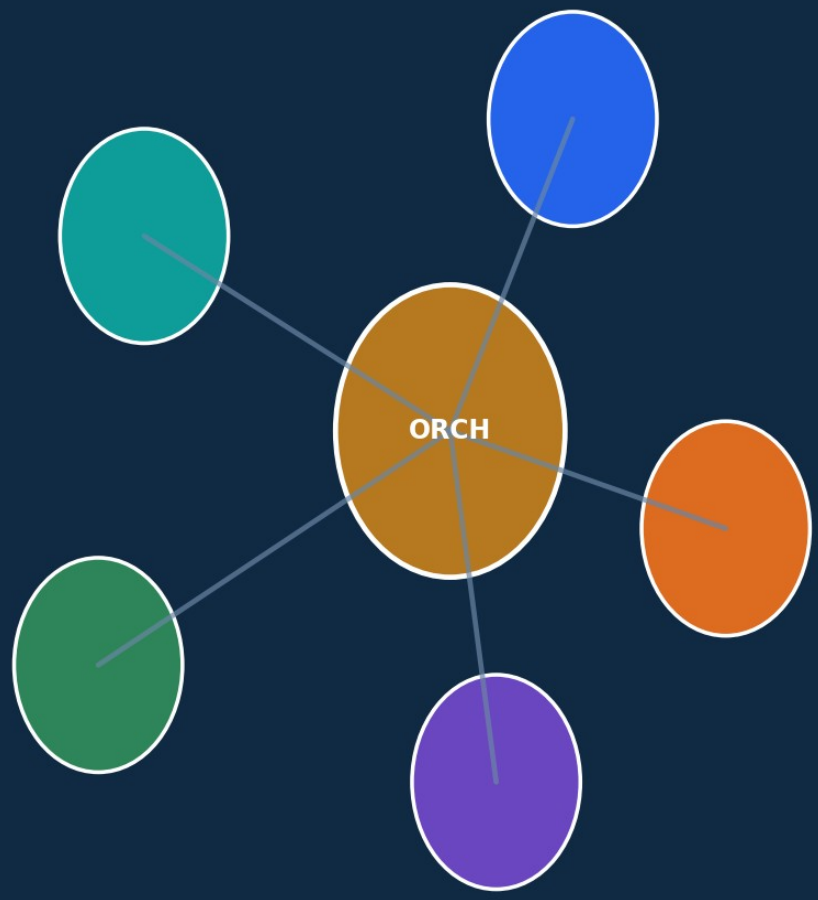


ENTERPRISE

AGENTIC SYSTEM PATTERNS

A practical catalog of orchestration, collaboration, event-driven, and human-oversight patterns

18 patterns | 36 detailed use cases | one infographic per pattern

Prepared for Bob Tichelman

22 October 2026

Current enterprise baseline across orchestration, collaboration, event-driven deployment, and human oversight.

EXECUTIVE ORIENTATION

Executive summary

The report catalogs 18 current enterprise patterns and 36 implementation-ready use cases. It treats "deployment pattern" broadly: how agents are triggered, how work and control move, how agents share state, and where people govern the result. The catalog includes the five multi-agent orchestrations built into Microsoft Agent Framework - sequential, concurrent, handoff, group chat, and Magentic - plus workflow, hierarchy, ensemble, event, shared-workspace, decentralized, and human-oversight patterns documented across the wider ecosystem. [S1]

Comprehensive enterprise baseline as of 22 October 2026. No standards body defines a closed, universally accepted list of agent patterns. This catalog reconciles current Microsoft Agent Framework and Azure guidance with patterns documented by Anthropic, Google ADK, AutoGen, LangChain/LangGraph, Amazon Bedrock, and current multi-agent research. The patterns are composable rather than mutually exclusive.

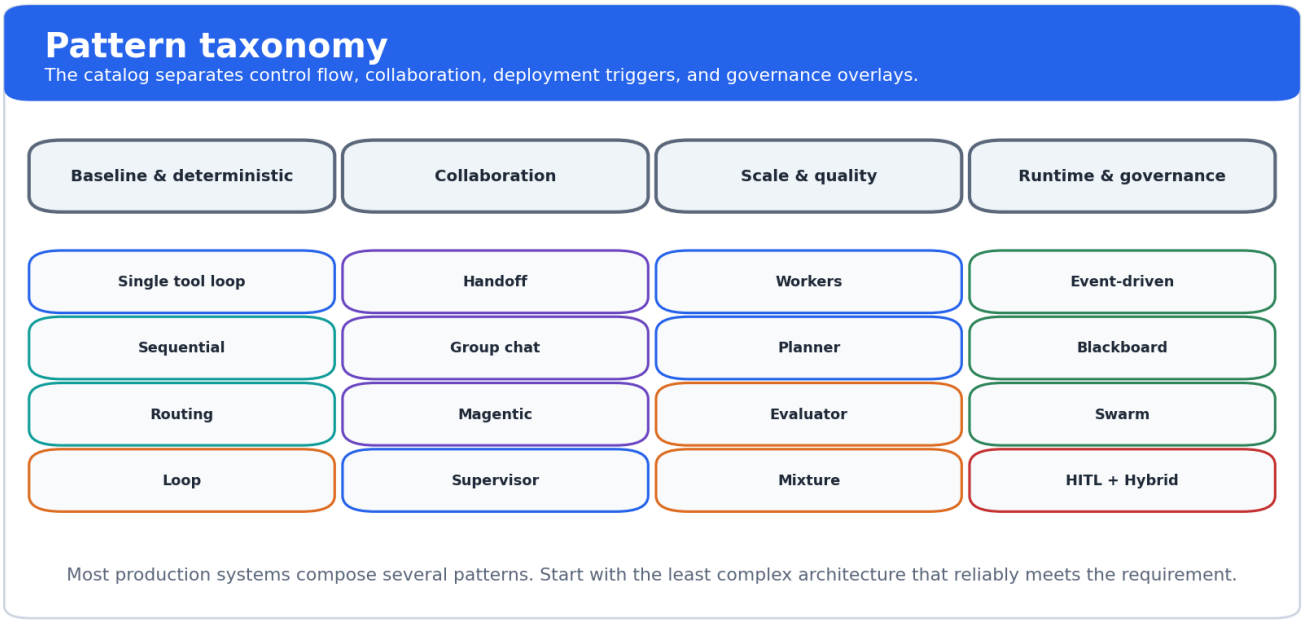


Figure 1. Practical taxonomy used in this report. The categories organize design decisions; they are not mutually exclusive.

Key conclusions

- Start with the least complex architecture that reliably meets the requirement. A single tool-using agent is often the correct production baseline; multi-agent coordination adds cost, latency, and failure modes. [S2]
- Separate deterministic control flow from model judgment. Use schemas, programmatic rules, and policy gates for money, identity, entitlements, calculations, and safety-critical actions.
- Treat human-in-the-loop as a risk-based overlay. Approval must be durable, attributable, evidence-rich, and capable of reject, edit, cancel, timeout, and escalation - not merely a confirmation click. [S14] [S15]
- Event Grid and Event Hubs are inputs and coordination infrastructure, not interchangeable agent patterns: Event Grid is best for discrete events; Event Hubs is best for high-volume streams. [S11]
- Magentic is the correct current term for the manager-led, ledger-driven adaptive pattern. Use it for open-ended work, not where a fixed or regulated workflow is sufficient. [S6] [S7]
- In production, broad systems are usually composite: route the request, select a workflow, parallelize independent work, apply review or approval gates, and preserve one end-to-end trace.

PATTERN SELECTION

Rapid selection guide

Choose the simplest row whose control flow and governance match the work. Add another pattern only when it solves a specific, measured limitation.

Pattern	Strongest fit	Non-negotiable control
Single-agent tool loop	One bounded domain; one coherent toolset	Tool-level approvals; iteration cap
Sequential	Known ordered stages and dependencies	Schema gates; checkpoints
Concurrent	Independent slices or expert perspectives	Aggregation and conflict rules
Routing	Distinct intent, risk, language, or cost classes	Confidence threshold; fallback route
Handoff	Ownership changes as context evolves	Transfer contract; cycle detection
Moderated group chat	Debate, review, consensus, maker-checker	Turn cap; completion rubric
Magentic	Open-ended research and adaptive collaboration	Plan signoff; stall/reset limits
Supervisor	One front door with bounded specialist domains	Central policy; scoped context
Orchestrator-workers	Unknown number of input-dependent subtasks	Spawn, queue, and budget limits
Planner-executor	Long-running task with inspectable plan	Plan approval; checkpoint/rollback
Deterministic loop	Machine-checkable repeat-until condition	Hard stop; idempotence
Evaluator-optimizer	Qualitative output with explicit rubric	Iteration cap; independent evaluator
Mixture of Agents	High-value synthesis benefiting from diversity	Diversity and aggregation rubric
Event-driven	Asynchronous events or streaming telemetry	Idempotency; replay; dead-letter
Blackboard	Long-lived shared evidence and tasks	Typed writes; provenance; conflict control
Swarm	Decentralized peers and dynamic availability	Peer identity; loop/global budget
Human-in-the-loop	High-impact action or ambiguity	Durable pause; decision rights
Adaptive hybrid	Broad platform or end-to-end process	Common contracts, policy, and tracing

Decision heuristics

- Known path? Use sequential. Unknown path but decomposable? Use orchestrator-workers or planner-executor. Open-ended and collaborative? Consider Magentic.
- Independent work? Use concurrent. One destination? Use routing. Ownership changes during the conversation? Use handoff.
- Need visible debate? Use moderated group chat. Need a single front door over bounded domains? Use supervisor.
- Need repeat-until control? Use a deterministic loop. Need qualitative critique? Use evaluator-optimizer.
- Need diverse layered reasoning? Use Mixture of Agents. Need durable artifact-centered collaboration? Use blackboard.
- Need asynchronous reaction? Use event-driven. Need decentralized local control? Use swarm cautiously.

- High-impact action, ambiguity, or regulated judgment? Add HITL regardless of the underlying pattern.

REPORT MAP

Chapters

Each chapter includes a definition, a unique infographic, fit and anti-fit guidance, control characteristics, and two detailed use-case requirement blueprints.

01. Single-Agent Tool Loop Baseline architecture	**02. Sequential / Pipeline** Deterministic workflow
03. Concurrent / Fan-Out-Fan-In Parallel workflow	**04. Routing / Triage** Decision workflow
05. Handoff / Transfer of Control Conversational collaboration	**06. Moderated Group Chat** Collaborative conversation
07. Magentic Dynamic Collaboration Open-ended collaboration	**08. Supervisor / Hierarchical Delegation** Centralized multi-agent
09. Orchestrator-Workers Dynamic decomposition	**10. Planner-Executor** Plan-then-act workflow
11. Deterministic Loop Controlled iteration	**12. Evaluator-Optimizer / Reflection** Quality-control loop
13. Mixture of Agents Layered ensemble	**14. Event-Driven / Reactive Agents** Deployment and trigger pattern
15. Blackboard / Shared Workspace Shared-state collaboration	**16. Swarm / Peer-to-Peer** Decentralized collaboration
17. Human-in-the-Loop (HITL) Governance overlay	**18. Adaptive Hybrid / Composite Orchestration** Meta-pattern

Single-Agent Tool Loop

Also known as: tool-using agent; ReAct-style agent; autonomous tool loop

One agent interprets the goal, chooses among approved tools and knowledge sources, observes the results, and repeats until it can answer or act. It is the preferred baseline when one domain, one security boundary, and one coherent toolset are sufficient. The loop can be autonomous, but production designs should bound iterations and high-impact actions. [S2] [S3]

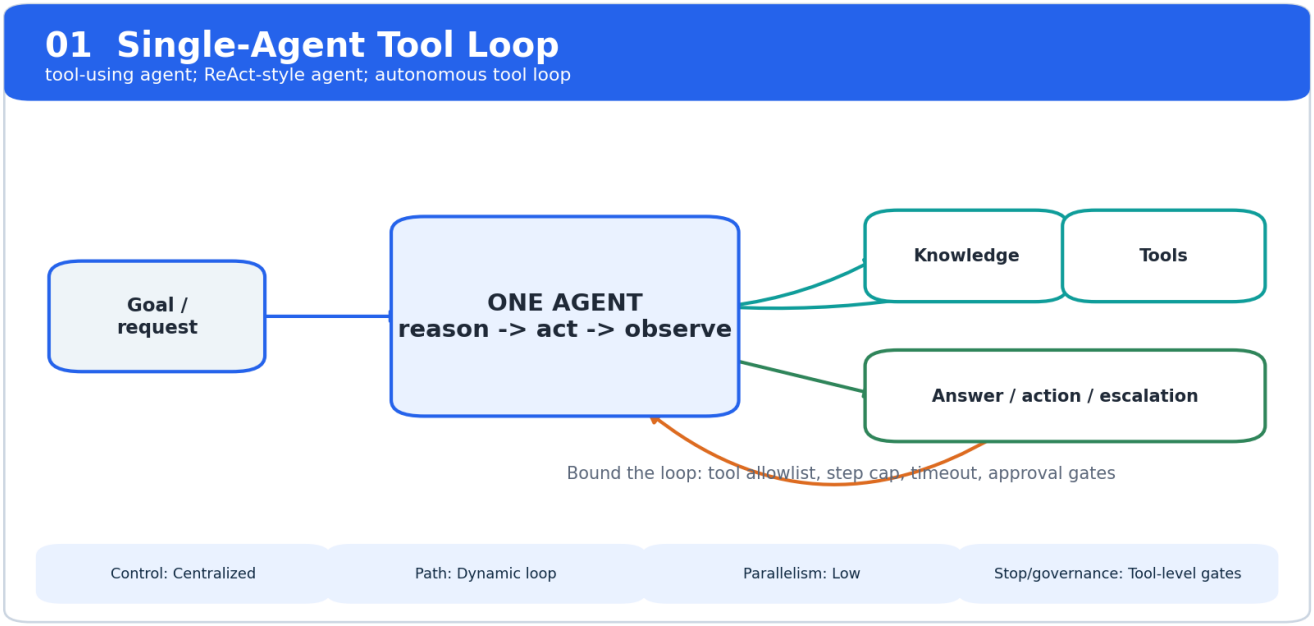


Figure 2. Single-Agent Tool Loop - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Centralized	**Dynamic loop**	**Low**	**Medium-high**	**Tool-level gates**

USE WHEN

A bounded domain with a manageable tool catalog, modest context, and no need for distinct agent identities or separate security boundaries.

AVOID WHEN

Tool overload, conflicting domain instructions, cross-functional ownership, or tasks that benefit from independent verification or parallel work.

How it works

1. Receive a goal and current context
2. Reason over the next best action
3. Call a permitted tool or retrieval source
4. Observe and update working state
5. Stop on success, budget exhaustion, or escalation

Use case 1.1: Employee policy and benefits assistant

Outcome: Answer employee questions, retrieve authoritative policy sections, calculate simple eligibility rules, and create a service ticket when the request cannot be resolved.

Requirement	Detailed specification
Trigger and cadence	Teams or web-chat message; optional voice transcript; explicit employee request to open a ticket.
Required inputs and data	Authenticated employee profile and locale; HR policy library; benefits plan documents; holiday calendar; case history; approved FAQ and structured eligibility tables.
Agent and human roles	One HR service agent with retrieval, calculation, and ticket-creation tools; no autonomous policy interpretation beyond approved rules.
Tools, APIs, and connectors	Microsoft Graph or chat channel; SharePoint/knowledge index; HRIS read API; ServiceNow or Dynamics case API; identity and consent service.
State and memory	Conversation state, retrieved citations, eligibility calculation inputs, and ticket draft; short retention for chat, longer retention only in the case system.
Security and governance	Employee-level authorization; PII masking in traces; country-specific data boundaries; approval before revealing sensitive records or filing a case on someone else's behalf.
Operations, evaluation, and recovery	p95 response target under 8 seconds for retrieval-only turns; maximum tool-call count; groundedness and deflection metrics; fallback to a human queue on low confidence or conflicting policies.
Required outputs	Cited answer, calculation explanation, next steps, or a confirmed ticket number with a concise handoff summary.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 1.2: Cloud resource diagnostic assistant

Outcome: Investigate a single subscription or application, correlate health and configuration data, propose remediation, and execute only preapproved low-risk runbooks.

Requirement	Detailed specification
Trigger and cadence	Engineer prompt; Azure Monitor alert link; incident record; scheduled health check.
Required inputs and data	Resource Graph inventory; Azure Monitor metrics and logs; change history; service health; architecture runbooks; CMDB ownership; incident context.
Agent and human roles	One operations agent with read tools plus a narrow catalog of idempotent remediation runbooks.
Tools, APIs, and connectors	Azure Resource Graph, Log Analytics, Monitor, Service Health, DevOps/GitHub, ITSM, Automation or Functions; managed identity per environment.
State and memory	Investigation notebook, evidence snapshots, hypotheses, tool results, and remediation status keyed by incident correlation ID.
Security and governance	Read-only default; environment-scoped RBAC; production changes require approval; command allowlist; secrets in Key Vault; redact tokens and customer data.
Operations, evaluation, and recovery	Tool timeouts and circuit breakers; retry only safe reads; evaluate diagnosis precision and mean time to resolution; preserve evidence when remediation fails.
Required outputs	Evidence-backed diagnosis, ranked remedies, approval request when needed, executed runbook receipt, and ITSM update.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S2] [S3] [S17]

Sequential / Pipeline

Also known as: prompt chaining; pipeline; linear delegation; pipes and filters

Agents execute in a predefined order, and each stage consumes the prior stage's output. The pattern makes dependencies, checkpoints, and accountability explicit, trading latency for specialized processing and progressive refinement. [S1] [S2] [S3]

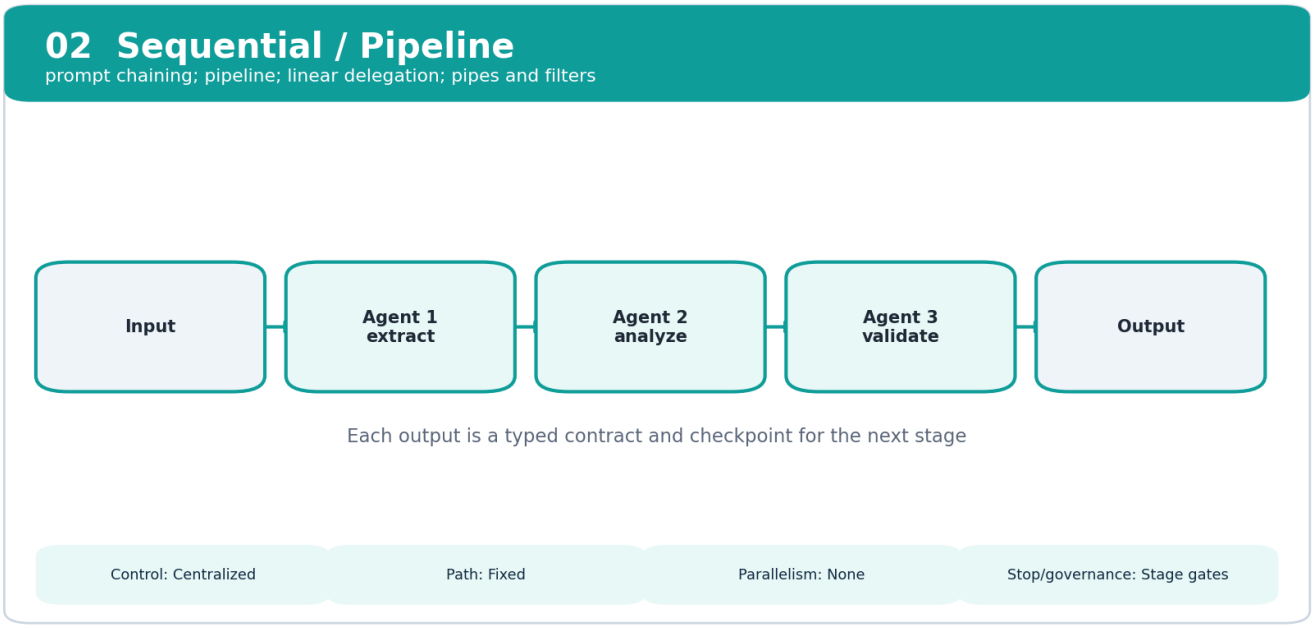


Figure 3. Sequential / Pipeline - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Centralized	**Fixed**	**None**	**Low-medium**	**Stage gates**

USE WHEN	AVOID WHEN
Stable processes with clear stage dependencies, predictable transformations, and well-defined input/output schemas.	Independent steps that should run in parallel, unknown task paths, frequent backtracking, or intermediate results that require dynamic routing.

How it works

1. Define the fixed stage order
2. Validate the output contract at every boundary
3. Pass only required context forward
4. Checkpoint durable state between long steps
5. Stop, retry, or compensate on failure

Use case 2.1: Monthly financial close narrative

Outcome: Transform approved close data into a reconciled variance analysis, management narrative, reviewer-approved commentary, and publishable reporting package.

Requirement	Detailed specification
Trigger and cadence	Close calendar milestone; controller marks source ledgers final; scheduled workflow after consolidation completes.
Required inputs and data	General ledger extracts; budget and forecast; prior-period actuals; FX rates; variance thresholds; chart of accounts; commentary templates; approved accounting policies.
Agent and human roles	Data validation agent -> variance analysis agent -> narrative drafting agent -> policy/compliance checker -> controller review gate -> publishing agent.
Tools, APIs, and connectors	ERP and planning APIs; governed semantic model/Power BI; secure file store; workflow approvals; Teams/email notification; optional Fabric notebook for calculations.
State and memory	Immutable close snapshot, stage outputs, reconciliations, reviewer comments, version lineage, and publication status.
Security and governance	Read-only ledger access; SOX segregation of duties; deterministic calculations outside the LLM; materiality rules; controller approval before distribution; full evidence trail.
Operations, evaluation, and recovery	Schema checks at every stage; reconcile totals to source; halt on data drift; rerun from checkpoints; evaluate numeric fidelity, citation coverage, and reviewer edit distance.
Required outputs	Variance table, cited management narrative, exceptions list, approval record, and final report package.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 2.2: Customer contract intake and risk review

Outcome: Extract commercial terms, compare them with playbooks, assess risk, generate redlines, and route the package for legal approval.

Requirement	Detailed specification
Trigger and cadence	Contract uploaded to SharePoint or CLM; inbox message to the contracts mailbox; CRM opportunity enters negotiation.
Required inputs and data	DOCX/PDF contract and attachments; OCR text; clause library; fallback positions; customer and opportunity metadata; jurisdiction; security and privacy addenda.
Agent and human roles	Ingestion/OCR agent -> clause extraction agent -> deviation agent -> risk scoring agent -> redline drafting agent -> legal approval.
Tools, APIs, and connectors	SharePoint/OneDrive, Exchange shared inbox, CRM, contract lifecycle system, document conversion/OCR, e-signature, and approval service.
State and memory	Source hash, extracted clause schema, comparison results, risk score, proposed edits, approvals, and final negotiated version.
Security and governance	Matter-based access; legal privilege labels; no training on customer documents; human approval for every redline; jurisdiction-specific playbooks; retention and legal hold.
Operations, evaluation, and recovery	Fail closed on unreadable pages or missing schedules; prevent stage skipping; compare extracted fields to golden sets; capture false-negative clause misses.
Required outputs	Structured term sheet, risk heat map, proposed redline, unresolved questions, and approved CLM record.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S1] [S2] [S3] [S4]

Concurrent / Fan-Out-Fan-In

Also known as: parallelization; scatter-gather; map-reduce; ensemble

Multiple agents work independently at the same time, either on separate slices of a task or from different expert perspectives. Their results are returned independently or combined by an aggregator using rules, voting, scoring, or synthesis. [S1] [S2] [S3]

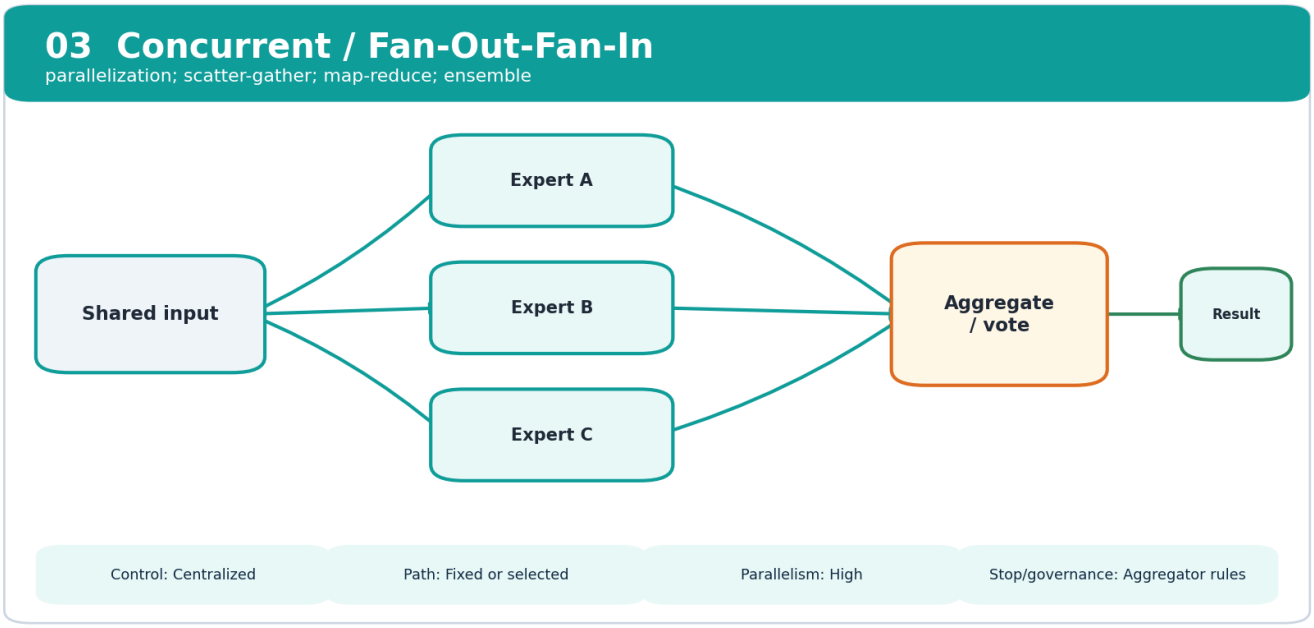


Figure 4. Concurrent / Fan-Out-Fan-In - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Centralized	**Fixed or selected**	**High**	**Medium**	**Aggregator rules**

USE WHEN	AVOID WHEN
Independent subtasks, time-sensitive analysis, broad coverage, guardrail separation, or scenarios that benefit from diverse perspectives.	Strong sequential dependencies, shared mutable state, no conflict-resolution method, or when aggregation would erase important disagreement.

How it works

1. Fan out a shared or partitioned request
2. Run independent agents concurrently
3. Enforce per-branch time and cost budgets
4. Normalize and reconcile outputs
5. Fan in to a result or portfolio of results

Use case 3.1: Omnichannel brand and issue monitoring

Outcome: Detect emerging topics, sentiment shifts, misinformation, service issues, and reputation risks across channels with a unified evidence-backed brief.

Requirement	Detailed specification
Trigger and cadence	Continuous stream window; spike threshold; executive request; scheduled hourly or daily digest.
Required inputs and data	Licensed social media feeds, Reddit/forums, news and RSS, app-store reviews, customer survey text, contact-center transcripts, public web pages, product and campaign taxonomy.
Agent and human roles	Social agent, news agent, review agent, contact-center agent, safety/misinformation agent, and an aggregation agent.
Tools, APIs, and connectors	Feed APIs and webhooks; Azure Event Hubs for high-volume streams; Event Grid for discrete alerts; stream processing; language detection; search/vector index; Teams/Power BI.
State and memory	Deduplicated event store, topic clusters, source provenance, rolling baselines, alert history, and analyst dispositions.
Security and governance	Respect platform terms and licensing; minimize personal data; distinguish public claims from verified facts; analyst approval before external response; regional data controls.
Operations, evaluation, and recovery	Backpressure, partitioning, deduplication, late-event handling, confidence thresholds, source weighting, and false-positive review; monitor lag, coverage, and alert precision.
Required outputs	Real-time alert, channel-by-channel findings, trend chart data, source links, uncertainty notes, and recommended response options.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 3.2: Enterprise due-diligence scan

Outcome: Assess a target, supplier, or strategic partner simultaneously across financial, cyber, legal, ESG, sanctions, and market dimensions.

Requirement	Detailed specification
Trigger and cadence	New diligence case; procurement risk tier change; acquisition workstream milestone.
Required inputs and data	Audited financials, data-room files, security questionnaires, legal records, sanctions/watchlists, ESG reports, market research, news, internal vendor history, and scoring rubric.
Agent and human roles	Financial, cyber, legal, compliance, ESG, and market agents plus an evidence-normalizing synthesis agent.
Tools, APIs, and connectors	Virtual data room, document intelligence/OCR, licensed risk feeds, internal GRC/procurement systems, web research, and secure case workspace.
State and memory	Case-scoped source registry, agent findings, confidence, evidence citations, conflicting claims, score components, and reviewer decisions.
Security and governance	Deal-team access boundaries; source licensing; no unsupported adverse conclusions; legal review; sanctions data handling; immutable audit record.
Operations, evaluation, and recovery	Parallel timeout policy; partial-results mode; conflict reconciliation; calibrated scoring; measure coverage, evidence quality, false alarms, and time saved.
Required outputs	Risk matrix, evidence pack, unresolved questions, confidence-weighted recommendation, and workstream-specific actions.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S1] [S2] [S3]

Routing / Triage

Also known as: classifier-router; intent routing; semantic routing; dispatch

A router classifies the request and sends it to the best specialized agent, workflow, model, or human queue. It separates concerns and prevents every request from carrying every tool and instruction. Routing can be rule-based, model-based, semantic, or hybrid. [S3] [S8] [S19]

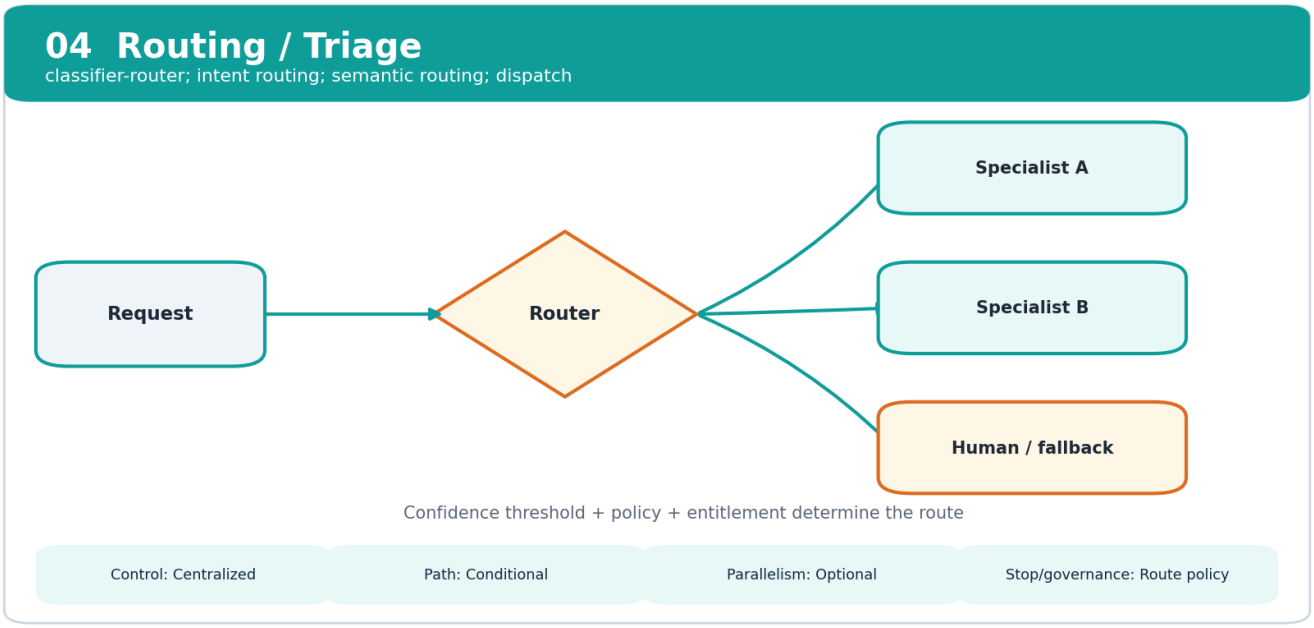


Figure 5. Routing / Triage - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Centralized	**Conditional**	**Optional**	**Low-medium**	**Route policy**

USE WHEN

Clearly distinguishable request types, business units, risk classes, languages, or model tiers with measurable routing accuracy.

AVOID WHEN

Highly ambiguous requests that need joint expertise, rapidly changing labels without feedback, or high-cost misroutes without a safe fallback.

How it works

1. Normalize and classify the request
2. Apply policy and entitlement checks
3. Select one or more destinations
4. Attach only destination-relevant context
5. Monitor misroutes and provide a fallback route

Use case 4.1: Enterprise service-desk triage

Outcome: Classify incoming requests, enrich them, resolve common issues automatically, and route the remainder to the correct resolver group with priority and context.

Requirement	Detailed specification
Trigger and cadence	Shared inbox message; Teams chat; portal form; voice transcript; monitoring alert; newly created ITSM ticket.
Required inputs and data	Inbox message body and attachments; requester identity and device inventory; service catalog; known errors; outage status; CMDB; support entitlements; urgency and impact rules.
Agent and human roles	Router, access agent, endpoint agent, application agent, network agent, service-request agent, and human queue fallback.
Tools, APIs, and connectors	Microsoft Graph for mail; Teams/bot channel; ITSM; Intune/endpoint management; CMDB; knowledge base; Event Grid for ticket-created events; notification service.
State and memory	Routing decision, confidence, extracted entities, enrichment data, attempts, resolution outcome, and corrected route from human agents.
Security and governance	Identity verification; attachment malware scanning; no password or secret handling in chat; privileged tasks require step-up authentication and approval; route-level RBAC.
Operations, evaluation, and recovery	Low-confidence threshold; no-match and multi-intent paths; route drift dashboard; evaluate top-1/top-3 routing accuracy, reassignment rate, and resolution time.
Required outputs	Resolved answer or action, enriched ticket, priority, assigned group, and a concise diagnostic summary.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 4.2: Sales and partner inquiry router

Outcome: Direct inbound requests to the appropriate product, region, account owner, partner program, or compliance review while preserving context.

Requirement	Detailed specification
Trigger and cadence	Website form, campaign response, sales mailbox, webinar question, partner portal submission, or CRM lead creation.
Required inputs and data	Message text; sender/company; country; language; product interest; account hierarchy; territory rules; existing opportunities; consent; restricted-party and industry flags.
Agent and human roles	Router plus product specialists, licensing agent, partner agent, security/compliance agent, and human seller queue.
Tools, APIs, and connectors	Marketing automation, Exchange inbox, CRM, partner system, territory service, identity resolution, calendar scheduling, and approved content library.
State and memory	Lead/contact match, route confidence, selected owner, duplicate detection, consent evidence, and follow-up SLA.
Security and governance	Marketing consent, data residency, fair routing, restricted-party screening, no autonomous commitments on pricing or legal terms, and human review for strategic accounts.
Operations, evaluation, and recovery	Fallback to sales operations; monitor misroutes, duplicate rates, response SLA, and conversion by route; retrain or revise taxonomy from accepted corrections.
Required outputs	Assigned owner/workflow, CRM enrichment, compliant acknowledgment, suggested response, and routing rationale.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S3] [S8] [S19]

Handoff / Transfer of Control

Also known as: agent transfer; delegation; escalation; peer handoff

The active agent transfers control to another agent when context, expertise, permissions, or risk changes. Unlike routing, which generally decides at intake, handoffs can occur repeatedly during execution and the receiving agent becomes the active conversational owner. [S1] [S2] [S4] [S8]

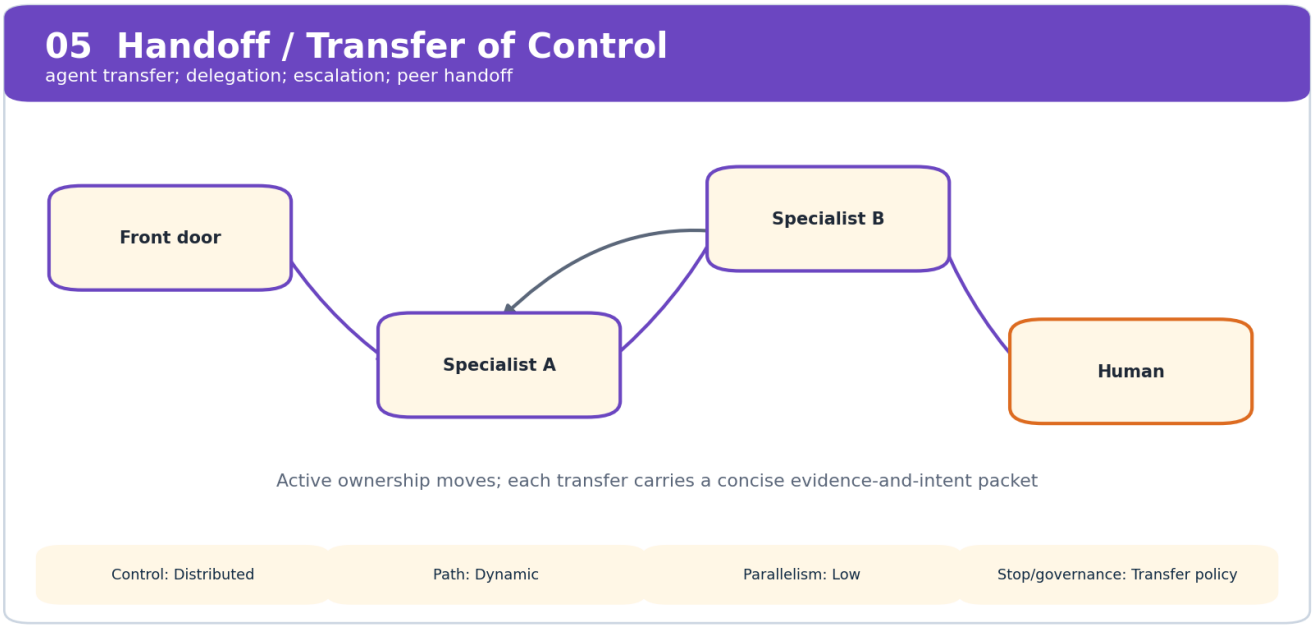


Figure 6. Handoff / Transfer of Control - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Distributed	**Dynamic**	**Low**	**Medium-high**	**Transfer policy**

USE WHEN

Conversations that cross domains over time, require specialist takeover, or need an explicit escalation path while retaining continuity.

AVOID WHEN

Tasks requiring simultaneous collaboration, weak ownership rules, uncontrolled handoff cycles, or situations where the receiver cannot access the required evidence.

How it works

1. Detect a transfer condition
2. Package a minimal handoff summary
3. Authorize the receiving agent
4. Transfer active ownership
5. Allow return, escalation, or termination

Use case 5.1: Member or patient service escalation

Outcome: Resolve administrative questions and transfer safely to benefits, clinical, pharmacy, billing, or human staff when the issue crosses a boundary.

Requirement	Detailed specification
Trigger and cadence	Authenticated chat or call transcript; detected clinical language; repeated failed resolution; explicit request for a person.
Required inputs and data	Member identity and consent; plan benefits; claims status; provider directory; prior interactions; approved clinical escalation criteria; language and accessibility needs.
Agent and human roles	Front-door service agent with handoffs to benefits, claims, pharmacy, billing, nurse line, crisis line, and human representative.
Tools, APIs, and connectors	Contact center, CRM/case management, benefits and claims APIs, provider directory, secure messaging, identity verification, and emergency escalation service.
State and memory	Shared conversation plus a structured transfer packet: intent, facts, completed checks, unresolved question, risk flags, and consent status.
Security and governance	No diagnosis by administrative agents; PHI minimum necessary; emergency language triggers immediate human protocol; recorded transfer reason; agent-specific access scopes.
Operations, evaluation, and recovery	Cycle detection, transfer limits, warm-handoff confirmation, human availability checks, and fallback callback workflow; measure transfer accuracy and repeat-story rate.
Required outputs	Resolved request or warm transfer with summary, case record, promised next step, and service-level clock.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 5.2: Security incident response handoffs

Outcome: Move an investigation among identity, endpoint, network, cloud, malware, and communications specialists as evidence changes.

Requirement	Detailed specification
Trigger and cadence	SIEM incident, Event Grid notification, analyst prompt, endpoint alert, or threat-intelligence match.
Required inputs and data	SIEM events, endpoint telemetry, identity logs, network flows, cloud activity, asset criticality, threat intelligence, prior incidents, and response playbooks.
Agent and human roles	Triage agent with peer handoffs to identity, endpoint, cloud, malware-analysis, containment, and communications agents; incident commander remains human.
Tools, APIs, and connectors	Microsoft Sentinel/SIEM, Defender/XDR, Event Hubs for telemetry, SOAR, ITSM, CMDB, threat-intel platforms, and secure collaboration channel.
State and memory	Incident graph, evidence chain, current owner, hypotheses, actions taken, approvals, and containment status.
Security and governance	Read-only investigation by default; containment approval based on asset tier; chain-of-custody controls; no evidence alteration; privileged actions through audited runbooks.
Operations, evaluation, and recovery	Handoff timeouts, owner heartbeat, severity-based escalation, duplicate-event correlation, and after-action evaluation of missed signals and unnecessary transfers.
Required outputs	Current incident owner, evidence-backed assessment, containment request or action receipt, stakeholder update, and timeline.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S1] [S2] [S4] [S8]

Moderated Group Chat

Also known as: moderator; group chat; roundtable; council; multi-agent debate

Specialized agents collaborate in a shared conversation while a moderator or chat manager selects who speaks next, enforces turn rules, and decides when the objective is complete. The moderator can be rule-based, model-based, or human. This is the direct match for the 'moderator' pattern. [S2] [S5] [S18]

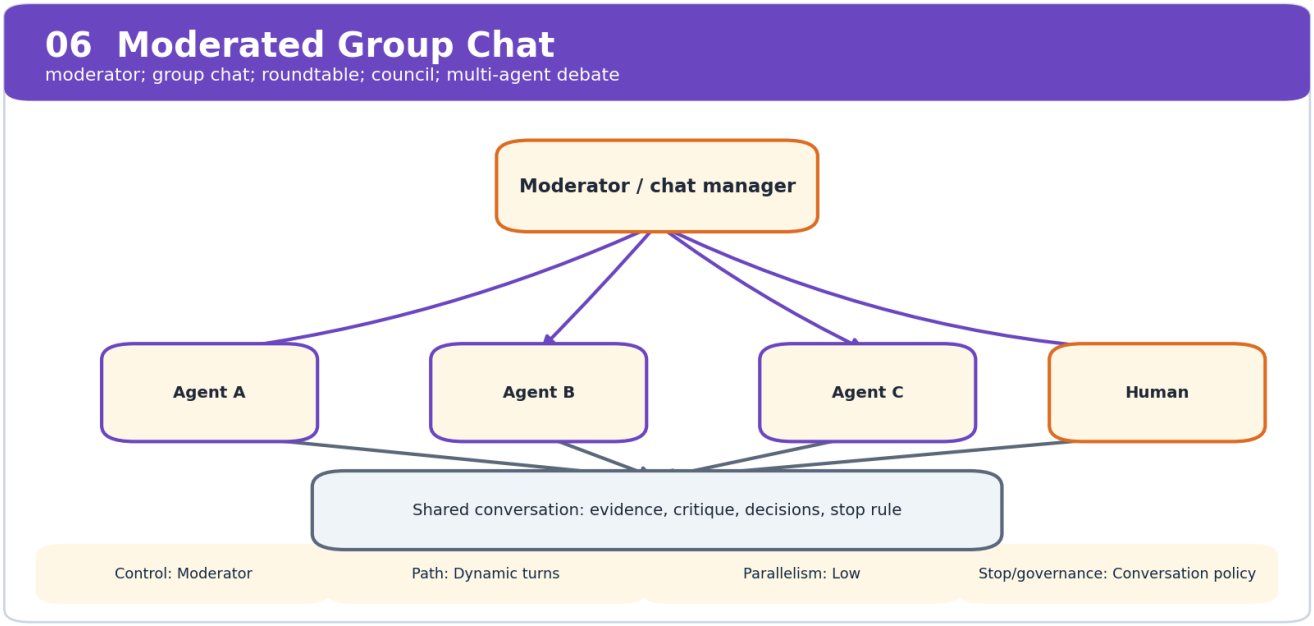


Figure 7. Moderated Group Chat - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Moderator	**Dynamic turns**	**Low**	**Medium**	**Conversation policy**

USE WHEN

Brainstorming, debate, cross-functional review, maker-checker work, or decisions that benefit from transparent iterative discussion.

AVOID WHEN

Real-time low-latency actions, large agent groups, no objective stopping rule, sensitive context that should not be visible to every participant, or simple delegation.

How it works

1. Place agents in one shared thread
2. Let the moderator choose the next speaker
3. Make evidence and critiques visible to all
4. Enforce turn, role, and completion rules
5. Synthesize consensus and preserve dissent

Use case 6.1: Architecture review board simulation

Outcome: Challenge a solution proposal from security, reliability, data, cost, operations, and product perspectives and produce a decision-ready review.

Requirement	Detailed specification
Trigger and cadence	Architecture document submitted; design-review meeting; major change request.
Required inputs and data	Architecture diagrams, requirements, threat model, data classification, workload estimates, budget, SLOs, standards, exception history, and open questions.
Agent and human roles	Moderator; security, reliability, data, cost, operations, and business-value agents; optional human architect as chair.
Tools, APIs, and connectors	SharePoint/OneDrive, diagram and document parsers, Azure inventory/cost APIs, policy repository, decision log, and meeting workspace.
State and memory	Shared transcript, claims and evidence, unresolved objections, decisions, action items, and stop criteria.
Security and governance	Read-only tool access; role-scoped evidence where needed; moderator prevents one agent from dominating; human owns acceptance and exceptions; record minority opinions.
Operations, evaluation, and recovery	Maximum rounds and speaking turns; question queue; factuality checks; completion rubric; evaluate issue coverage, duplicate objections, and decision usefulness.
Required outputs	Decision recommendation, approved conditions, rejected options, risks, owners, due dates, and traceable rationale.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 6.2: Crisis communications review room

Outcome: Create and stress-test internal and external communications during a fast-moving incident without allowing agents to publish directly.

Requirement	Detailed specification
Trigger and cadence	Major outage, safety event, security incident, social-media spike, or executive activation.
Required inputs and data	Verified incident facts; operational timeline; customer impact; legal guidance; support volumes; social media feeds; news coverage; approved tone and templates.
Agent and human roles	Moderator; incident-facts, legal, customer, employee, social-listening, accessibility, and executive-voice agents; human communications lead.
Tools, APIs, and connectors	Incident system, contact center, licensed social feeds, news/RSS, intranet, status-page draft API, email drafting, and approval workflow.
State and memory	Single source of verified facts, rumor register, draft versions, objections, audience variants, approvals, and publication status.
Security and governance	Agents cannot publish; facts must be source-tagged; legal and communications approval; embargo and insider-information controls; preserve review transcript.
Operations, evaluation, and recovery	Short turn timer; stale-fact detection; conflicting-source flag; maximum revision rounds; evaluate time to approved draft, corrections after publication, and audience clarity.
Required outputs	Approved message variants, FAQ, holding statement, executive brief, unresolved fact list, and publication checklist.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY Authoritative sources, access, freshness, schema, and provenance are verified.	**SAFE TO RUN** Least privilege, action gates, stop rules, and human fallback are tested.	**OPERABLE** Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S2] [S5] [S18]

Magentic Dynamic Collaboration

Also known as: Magentic; Magentic-One-style manager; ledger-driven orchestration

A powerful manager coordinates specialized agents for complex, open-ended work. It builds a plan, tracks progress in a ledger, chooses the next agent from the evolving context, detects stalls, and replans. 'Magentic' is the current term; it is sometimes misheard or typed as 'magenta.' [S6] [S7]

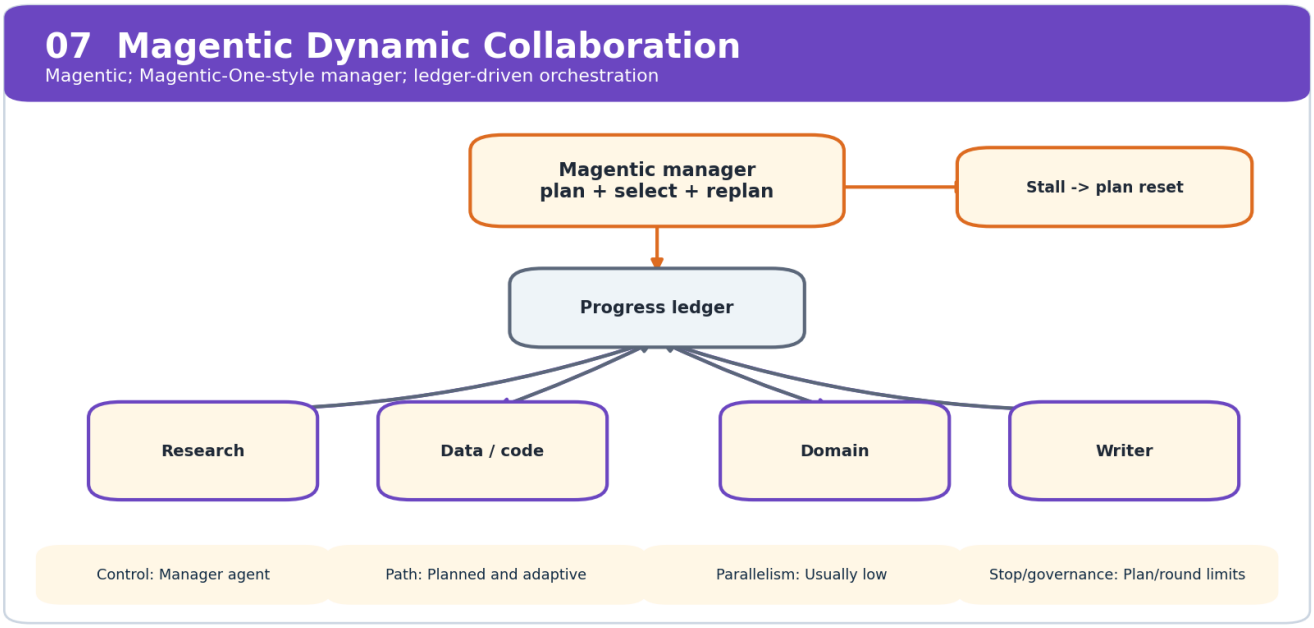


Figure 8. Magentic Dynamic Collaboration - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Manager agent	**Planned and adaptive**	**Usually low**	**High**	**Plan/round limits**

USE WHEN

Open-ended research, web/file tasks, analysis and computation where the solution path cannot be fully specified beforehand.

AVOID WHEN

Simple or regulated deterministic workflows, weakly sandboxed tools, tasks without reliable progress signals, or cases where model-driven planning cost and variance are unacceptable.

How it works

1. Create a fact sheet and initial plan
2. Select the next specialist dynamically
3. Track progress and unresolved facts
4. Detect stalls and reset or replan
5. Synthesize the final result, optionally after plan signoff

Use case 7.1: Open-ended market and competitor research brief

Outcome: Investigate a strategic question across web, licensed sources, internal files, data analysis, and scenario modeling when the needed steps are not known in advance.

Requirement	Detailed specification
Trigger and cadence	Executive research question; strategy offsite; product planning cycle; unexpected competitor announcement.
Required inputs and data	Research question and decision criteria; internal strategy files; product telemetry; CRM win/loss notes; licensed market data; public web/news; financial data; date and geography scope.
Agent and human roles	Magentic manager with web research, internal-research, data-analysis/code, financial-analysis, and synthesis agents.
Tools, APIs, and connectors	Enterprise search, web search, licensed data APIs, CRM, file repositories, sandboxed code interpreter, citation store, and report workspace.
State and memory	Fact sheet, plan, progress ledger, source registry, hypotheses, calculations, stalls, plan resets, and intermediate artifacts.
Security and governance	Plan review for sensitive questions; browser and code sandbox; source licensing; no unsupported competitor claims; model/tool budgets; human signoff before external use.
Operations, evaluation, and recovery	Maximum coordination rounds, consecutive-stall limit, plan-reset limit, checkpointing, and streaming progress; evaluate source quality, completeness, reproducibility, and decision relevance.
Required outputs	Cited brief, evidence table, calculations, scenarios, confidence and gaps, plus a record of the final plan and source set.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 7.2: Complex M&A data-room investigation

Outcome: Pursue cross-document diligence questions that evolve as new facts, inconsistencies, and required analyses emerge.

Requirement	Detailed specification
Trigger and cadence	Deal-team question; new data-room tranche; diligence finding that creates follow-up work.
Required inputs and data	Data-room files, financial models, contracts, HR summaries, technical architecture, cyber assessments, customer metrics, issue list, and deal thesis.
Agent and human roles	Manager plus document investigator, financial modeler, contract analyst, cyber analyst, commercial analyst, and report synthesizer.
Tools, APIs, and connectors	Virtual data room, OCR/document intelligence, secure code execution, spreadsheet parser, deal issue tracker, and encrypted case store.
State and memory	Matter-isolated ledger, source hashes, question tree, calculations, contradictions, privileged notes, approvals, and incomplete requests.
Security and governance	Strict deal-room membership; information barriers; privilege and watermarking; download restrictions; plan approval for broad searches; no source mutation.
Operations, evaluation, and recovery	Per-agent quotas; resumable checkpoints; missing-file requests; contradiction resolution; red-team evaluation for fabricated numbers and cross-deal leakage.
Required outputs	Diligence memo, issue register, source-linked findings, model outputs, unresolved requests, and management questions.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S6] [S7] [S2]

Supervisor / Hierarchical Delegation

Also known as: supervisor; manager-worker hierarchy; subagents; agent-as-tool

A supervisor remains the user-facing owner and invokes specialized subagents as tools or collaborators. Subagents normally return results to the supervisor instead of taking over the conversation. Hierarchies can be nested to isolate domains, context, and permissions. [S8] [S9]

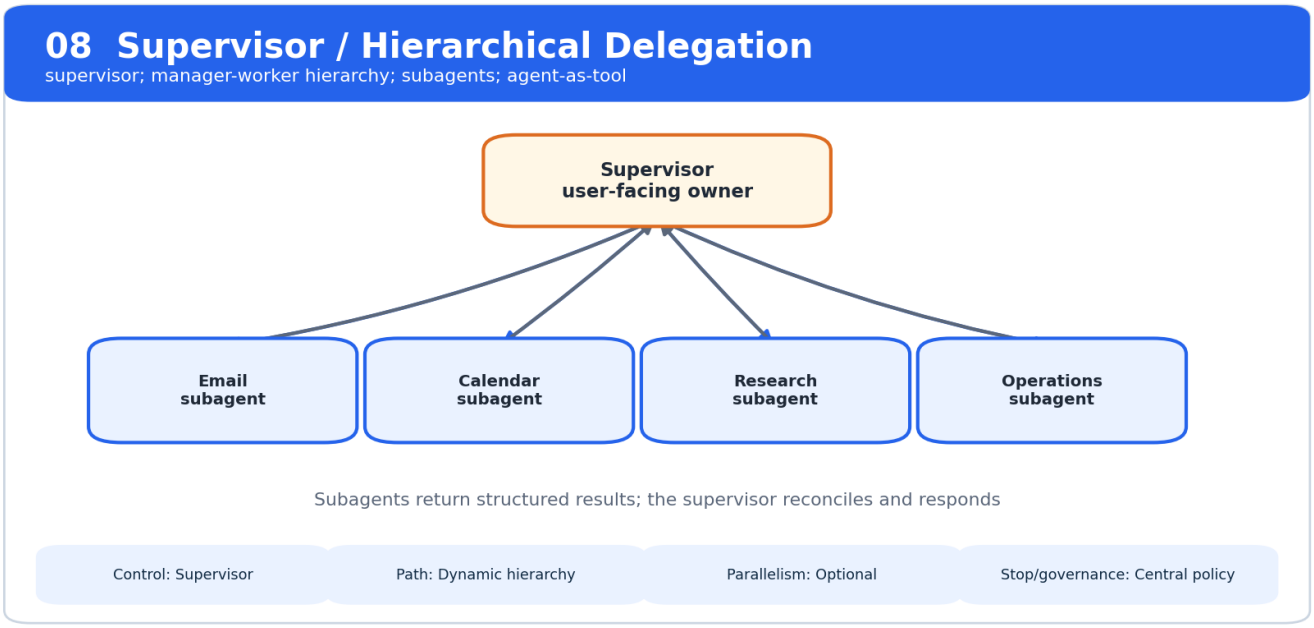


Figure 9. Supervisor / Hierarchical Delegation - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Supervisor	**Dynamic hierarchy**	**Optional**	**Medium-high**	**Central policy**

USE WHEN

One coherent front door, multiple well-bounded domains, centralized policy enforcement, and a need to keep specialist prompts or context isolated.

AVOID WHEN

Peer collaboration is primary, direct specialist-to-user takeover is required, the supervisor becomes a latency bottleneck, or domain boundaries are poorly defined.

How it works

1. Supervisor interprets the request
2. Select one or more specialist subagents
3. Provide scoped context and permissions
4. Receive structured results
5. Reconcile and respond through the supervisor

Use case 8.1: Enterprise executive assistant

Outcome: Provide one conversational front door that delegates email, calendar, travel, research, expense, and document work to domain agents.

Requirement	Detailed specification
Trigger and cadence	User prompt in Copilot/chat; inbox follow-up request; meeting reminder; scheduled daily brief.
Required inputs and data	Inbox messages, calendar events, contacts, Teams chats, travel preferences, expense policy, documents, task list, and explicit user instructions.
Agent and human roles	Supervisor with email, calendar, travel, research, expense, and document subagents; human user is final authority for sends, bookings, and submissions.
Tools, APIs, and connectors	Microsoft Graph, Teams, enterprise search, travel provider, expense system, task service, and approval UI; delegated user identity where appropriate.
State and memory	User preferences, pending commitments, subagent summaries, drafts, action status, consent, and provenance; isolate private contexts by tool and task.
Security and governance	Never send, book, purchase, or submit without confirmation; least-privilege delegated scopes; sensitivity-label enforcement; external-recipient and cost checks.
Operations, evaluation, and recovery	Parallelize independent reads; cap subagent fan-out; handle stale calendar or email state; evaluate task completion, incorrect actions prevented, latency, and user corrections.
Required outputs	Unified answer or brief, reviewable drafts, proposed schedule, approval cards, and confirmed action receipts.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY Authoritative sources, access, freshness, schema, and provenance are verified.	**SAFE TO RUN** Least privilege, action gates, stop rules, and human fallback are tested.	**OPERABLE** Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 8.2: Cloud operations command agent

Outcome: Coordinate domain-specific operations agents while preserving a single incident owner and policy layer.

Requirement	Detailed specification
Trigger and cadence	Operations prompt, major incident, service-health event, or scheduled posture review.
Required inputs and data	Application topology, alerts, logs, deployment history, ownership, SLOs, runbooks, cost data, security findings, and business criticality.
Agent and human roles	Supervisor with application, platform, network, database, security, cost, and communications subagents.
Tools, APIs, and connectors	Azure Monitor, Resource Graph, DevOps/GitHub, Kubernetes, ITSM, CMDB, FinOps platform, and approved automation.
State and memory	Incident or review state, specialist findings, conflicts, decisions, approvals, and action outcomes maintained by the supervisor.
Security and governance	Environment and domain RBAC; read-only specialists by default; production action approval; separation between diagnosis and remediation; audited commands.
Operations, evaluation, and recovery	Supervisor circuit breaker; partial-result handling; conflict reconciliation; nested timeout budgets; measure diagnosis accuracy, MTTR, action rollback, and cost per incident.
Required outputs	Unified diagnosis, prioritized actions, approval requests, stakeholder communication, and incident record updates.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S8] [S9] [S17]

Orchestrator-Workers

Also known as: dynamic task decomposition; coordinator-workers; map-and-synthesize

A central orchestrator decides at runtime which subtasks are needed, creates worker assignments, often executes them in parallel, and synthesizes the outputs. The worker set or task count is not fixed in advance. [S3] [S19]

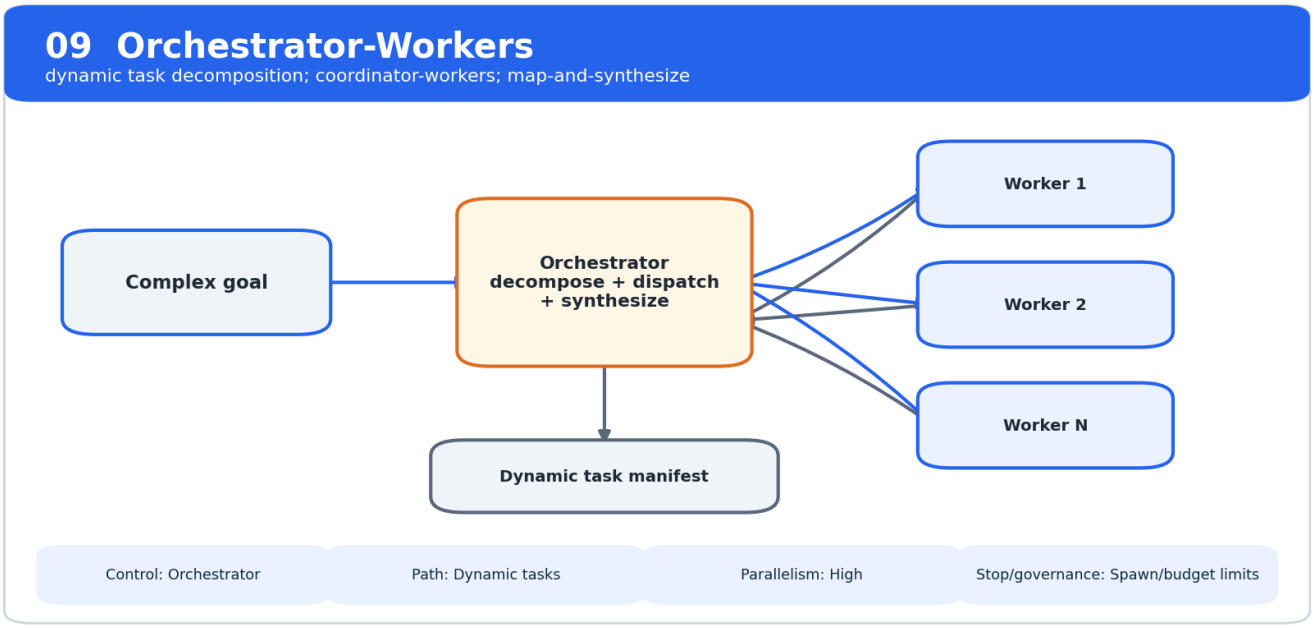


Figure 10. Orchestrator-Workers - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Orchestrator	**Dynamic tasks**	**High**	**High**	**Spawn/budget limits**

USE WHEN

Large tasks whose partitions depend on the input, such as repository changes, document collections, broad searches, or variable-scope analysis.

AVOID WHEN

Small fixed tasks, shared-state work that cannot be partitioned, expensive aggregation, or when dynamic spawning would violate control limits.

How it works

1. Inspect the goal and available artifacts
2. Decompose into runtime-defined subtasks
3. Create and dispatch worker assignments
4. Track completion and retry failed partitions
5. Synthesize and validate the combined result

Use case 9.1: Large contract-portfolio analysis

Outcome: Analyze an arbitrary collection of contracts by dynamically creating workers for documents, clauses, jurisdictions, and questions, then synthesize portfolio risk.

Requirement	Detailed specification
Trigger and cadence	Portfolio uploaded; regulatory change; acquisition; renewal campaign; legal discovery question.
Required inputs and data	Contract corpus and metadata; OCR text; clause taxonomy; regulatory rules; extraction schemas; risk thresholds; business owner and renewal dates.
Agent and human roles	Orchestrator; document workers; clause/jurisdiction specialists; exception validator; portfolio synthesis agent.
Tools, APIs, and connectors	Blob/SharePoint storage, document intelligence, queue or task runtime, vector/search index, contract system, analytics store, and review interface.
State and memory	Manifest, partition map, worker status, extracted records, duplicate groups, exceptions, lineage, and aggregate calculations.
Security and governance	Matter and document ACLs; worker identity isolation; no cross-tenant context; legal review for conclusions; immutable source hashes and extraction version.
Operations, evaluation, and recovery	Idempotent workers, queue backpressure, retry/dead-letter, skew handling for large documents, aggregation validation, and cost/throughput dashboards.
Required outputs	Contract inventory, clause matrix, risk clusters, renewal actions, exceptions, and source-linked portfolio report.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 9.2: Repository-wide software change

Outcome: Plan and implement a change across an unknown number of files while using workers for code areas, tests, documentation, and security review.

Requirement	Detailed specification
Trigger and cadence	Approved engineering issue, dependency upgrade, API migration, or vulnerability remediation.
Required inputs and data	Repository, issue requirements, architecture notes, coding standards, dependency graph, CI results, security policies, and change constraints.
Agent and human roles	Orchestrator; codebase mapping worker; per-module implementation workers; test worker; documentation worker; security reviewer; final integration agent.
Tools, APIs, and connectors	GitHub/Azure DevOps, code search, sandboxed build environment, package registries, CI/CD, static analysis, test service, and pull-request API.
State and memory	Worktree snapshot, task graph, branch/commit mapping, worker diffs, build/test results, conflicts, and review comments.
Security and governance	No direct push to protected branches; sandbox and network egress limits; dependency allowlist; secrets scanning; human code review and merge approval.
Operations, evaluation, and recovery	Limit worker count and changed files; rebase/conflict strategy; cancel on failing safety tests; evaluate build pass rate, regression rate, review effort, and rollback readiness.
Required outputs	Reviewable pull request, tests, migration notes, security findings, unresolved conflicts, and deployment/rollback plan.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY Authoritative sources, access, freshness, schema, and provenance are verified.	**SAFE TO RUN** Least privilege, action gates, stop rules, and human fallback are tested.	**OPERABLE** Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S3] [S19] [S16]

Planner-Executor

Also known as: plan-and-execute; task graph; plan-then-act

Planning and execution are separated. A planner produces a structured, reviewable plan or task graph; one or more executors perform the steps; the planner or a controller checks progress and may replan. This makes intent visible before actions occur. [S3] [S4] [S19]

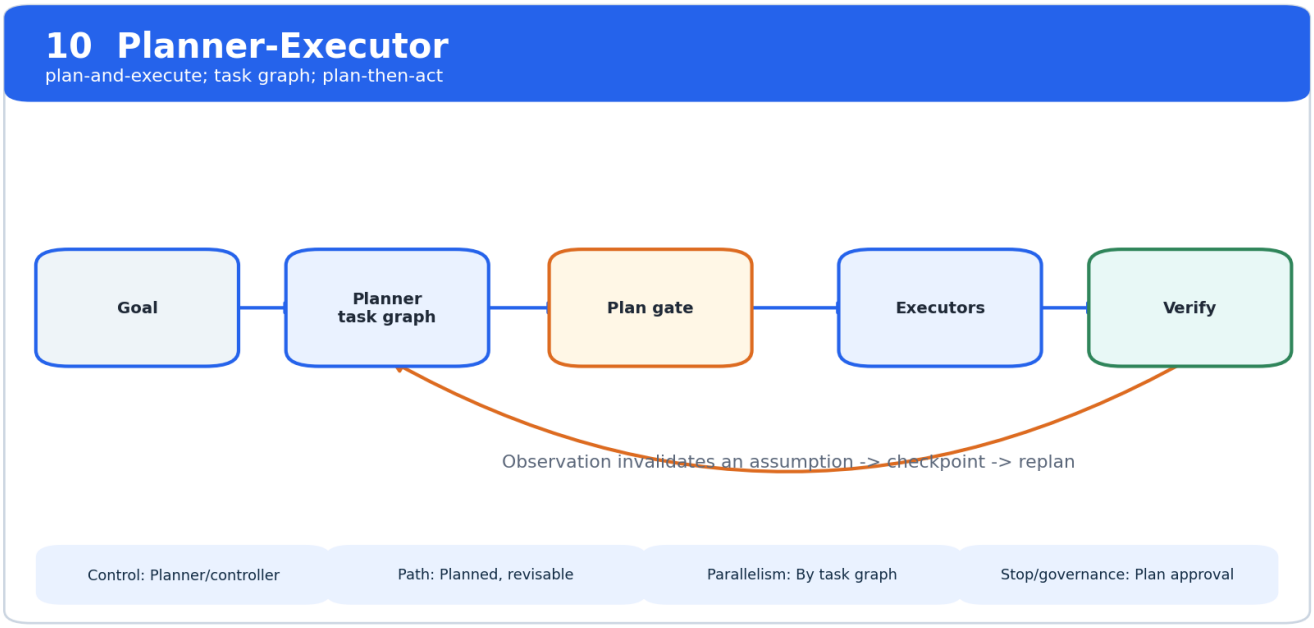


Figure 11. Planner-Executor - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Planner/controller	**Planned, revisable**	**By task graph**	**High**	**Plan approval**

USE WHEN	AVOID WHEN
Long-running tasks with meaningful dependencies, costly actions, coordination across systems, or a need to inspect the plan before execution.	Simple tasks, rapidly changing environments where plans become stale instantly, or domains where a plan cannot be validated before action.

How it works

1. Translate the goal into constraints
2. Create a structured plan and dependencies
3. Validate or approve the plan
4. Execute steps with checkpoints
5. Replan when observations invalidate assumptions

Use case 10.1: Data-center or cloud migration wave

Outcome: Build a dependency-aware migration plan, obtain approvals, execute automated steps, verify outcomes, and replan around failures.

Requirement	Detailed specification
Trigger and cadence	Migration wave approved; maintenance window; application owner readiness confirmation.
Required inputs and data	CMDB, dependency maps, discovery data, application criticality, target architecture, network and identity requirements, maintenance windows, runbooks, and rollback criteria.
Agent and human roles	Planner; discovery validator; network, identity, data, application, test, and cutover executors; human change authority.
Tools, APIs, and connectors	CMDB, Azure Migrate, infrastructure-as-code repository, DevOps pipeline, monitoring, ITSM/change management, DNS/network automation, and backup systems.
State and memory	Versioned plan graph, dependencies, approvals, step status, evidence, exceptions, rollback points, and replans.
Security and governance	Formal change approval; maintenance-window enforcement; environment RBAC; backup verification; separation of planning and privileged execution; emergency stop.
Operations, evaluation, and recovery	Precondition checks, idempotent steps, checkpoint/resume, compensating actions, critical-path monitoring, and metrics for migration success, downtime, and rollback frequency.
Required outputs	Approved plan, run-of-show, executed change record, validation evidence, replanned tasks, and closure report.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 10.2: Large customer event or training program planning

Outcome: Convert objectives and constraints into a coordinated plan covering content, speakers, facilities, accessibility, communications, and follow-up.

Requirement	Detailed specification
Trigger and cadence	Event charter approval; customer commitment; date or scope change.
Required inputs and data	Audience profiles, learning objectives, venue and virtual-platform constraints, speaker availability, budget, travel rules, content assets, accessibility needs, and risk register.
Agent and human roles	Planner with content, scheduling, logistics, travel, communications, accessibility, and evaluation executors; program owner approves commitments.
Tools, APIs, and connectors	Calendar, Teams, SharePoint, registration platform, travel system, procurement, email drafting, task tracker, and survey platform.
State and memory	Plan graph, dependencies, owners, milestones, budget assumptions, approvals, change log, and contingency branches.
Security and governance	No booking, purchase, invite, or external send without approval; attendee privacy; accessibility checklist; vendor and travel policy checks.
Operations, evaluation, and recovery	Critical-path alerts, stale-assumption detection, dependency updates, rescheduling policy, and evaluation against attendance, readiness, budget, and learning outcomes.
Required outputs	Approved plan, agenda, owner matrix, communications drafts, risk/contingency plan, and execution dashboard.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S3] [S4] [S19] [S14]

Deterministic Loop

Also known as: loop agent; repeat-until; iterative workflow

One or more steps repeat under explicit programmatic conditions, such as processing every item, retrying until a test passes, or iterating until a measurable threshold is reached. The stopping rule is part of the workflow, not left entirely to the model. [S4] [S19]

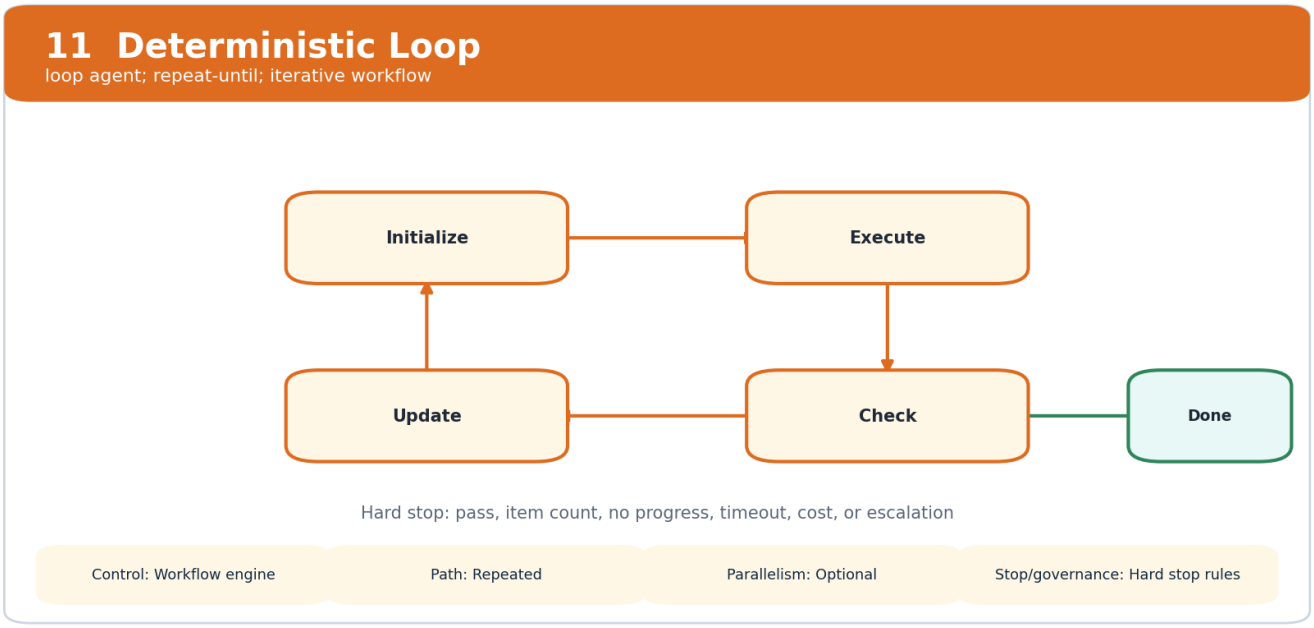


Figure 12. Deterministic Loop - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Workflow engine	**Repeated**	**Optional**	**Medium**	**Hard stop rules**

USE WHEN	AVOID WHEN
Batch processing, test-fix cycles, data-quality remediation, or any workflow with a clear machine-checkable stop condition.	Subjective quality without an evaluator, unlimited self-reflection, non-idempotent repeated actions, or high-cost steps with no progress measure.

How it works

1. Initialize state and loop counter
2. Execute the repeated step
3. Evaluate a deterministic condition
4. Update state or advance the item cursor
5. Stop at success, cap, timeout, or escalation

Use case 11.1: Data-quality repair cycle

Outcome: Profile a dataset, propose repairs, apply approved transformations, rerun quality checks, and continue until thresholds are met or the exception budget is exhausted.

Requirement	Detailed specification
Trigger and cadence	New file/table; failed quality gate; scheduled pipeline; schema-change event.
Required inputs and data	CSV/XLSX/table; schema; data dictionary; quality rules; reference/master data; permitted transformations; acceptance thresholds.
Agent and human roles	Profiler, repair planner, transformation executor, and deterministic quality checker in a bounded loop.
Tools, APIs, and connectors	Fabric/Databricks/Spark, storage, data catalog, master-data service, pipeline orchestrator, and issue tracker.
State and memory	Dataset version, issue inventory, proposed and applied transformations, quality scores by iteration, rejected fixes, and lineage.
Security and governance	Never overwrite source; PII controls; approved transformation allowlist; human review for ambiguous merges or deletions; reproducible code for every repair.
Operations, evaluation, and recovery	Maximum iterations, minimum score improvement, rollback per iteration, idempotence, and quarantine path; evaluate rule pass rate, precision of fixes, and data loss.
Required outputs	Cleaned dataset, transformation script, quality report, unresolved exceptions, and lineage record.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 11.2: Test-fix cycle for analytics or code

Outcome: Generate or modify an artifact, run deterministic tests, interpret failures, fix it, and repeat within strict budgets.

Requirement	Detailed specification
Trigger and cadence	Developer request; failed CI job; new analytics requirement; broken notebook or formula suite.
Required inputs and data	Source code/notebook/workbook, requirements, tests, sample data, style and security rules, runtime constraints, and dependency lock file.
Agent and human roles	Builder agent and deterministic test harness; optional specialist debugger for categorized failures.
Tools, APIs, and connectors	Sandbox, CI runner, unit/integration tests, linting, static analysis, package registry, source control, and artifact store.
State and memory	Iteration number, diff, test results, failure classification, build logs, resource use, and best-known passing artifact.
Security and governance	Sandboxed execution; network and file limits; no production deployment; license and secret scans; human review before merge or release.
Operations, evaluation, and recovery	Stop on pass, no-progress threshold, repeated failure signature, cost cap, or timeout; preserve last good state; measure pass rate and regressions.
Required outputs	Passing artifact or best attempt, full test evidence, change summary, remaining failures, and reviewable pull request/package.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S4] [S19] [S17]

Evaluator-Optimizer / Reflection

Also known as: maker-checker; generator-verifier; critic-reviser; reflection loop

A maker generates or revises an artifact while an evaluator assesses it against explicit criteria and returns actionable feedback. The cycle stops on acceptance or a hard iteration limit. Unlike a generic loop, the stop decision includes qualitative evaluation. [S2] [S3] [S19]

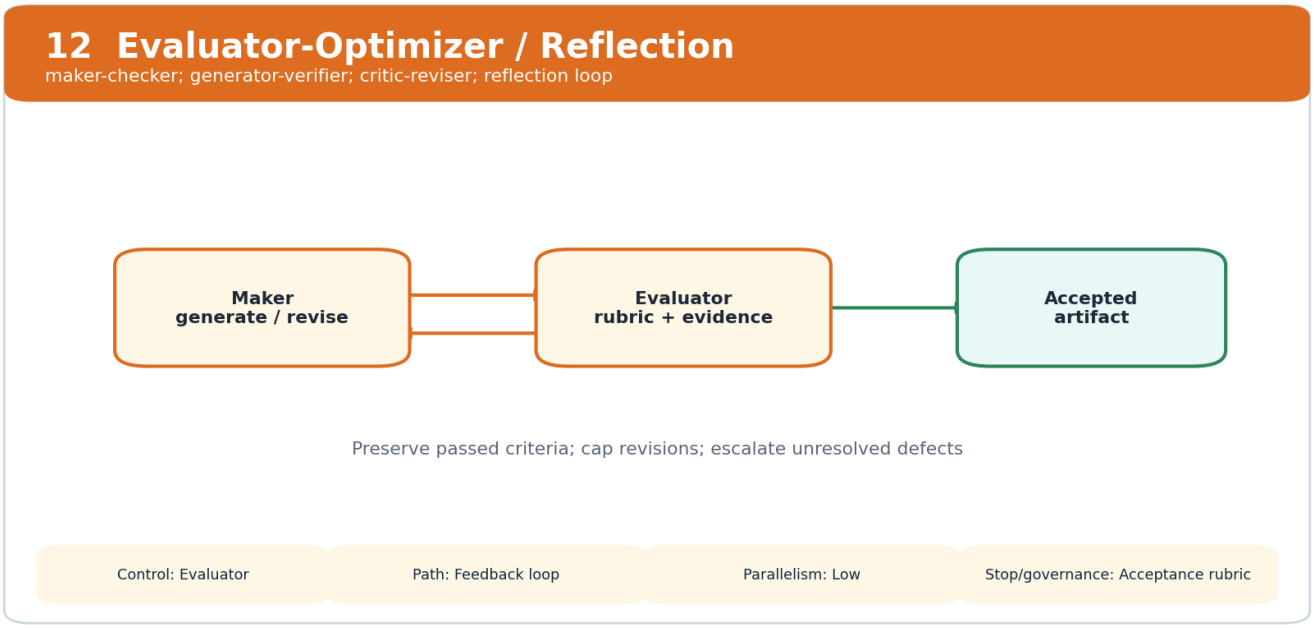


Figure 13. Evaluator-Optimizer / Reflection - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Evaluator	**Feedback loop**	**Low**	**Medium-high**	**Acceptance rubric**

USE WHEN	AVOID WHEN
Outputs that measurably improve through critique: writing, translation, policy checks, reasoning, code review, or compliance evidence.	No objective rubric, evaluator and maker share the same blind spot, marginal value is low, or repeated revisions can introduce new risk.

How it works

1. Generate a candidate
2. Evaluate against a rubric and evidence
3. Return specific defects and priority
4. Revise without losing accepted content
5. Stop on pass, cap, or human escalation

Use case 12.1: Regulated customer communication

Outcome: Draft a response and iteratively check it for factual grounding, required disclosures, tone, accessibility, and policy compliance before human approval.

Requirement	Detailed specification
Trigger and cadence	Complaint or appeal; adverse-action notice; regulated product inquiry; executive correspondence.
Required inputs and data	Customer message and case facts; approved product terms; policy and regulatory rules; mandatory disclosures; tone and reading-level standards; prior correspondence.
Agent and human roles	Drafting agent, factuality evaluator, compliance evaluator, and reviser; human approver remains final sender.
Tools, APIs, and connectors	CRM/case system, inbox, policy retrieval, template service, readability checker, approval workflow, and outbound drafting channel.
State and memory	Source facts, draft versions, rubric scores, evaluator comments, accepted clauses, open defects, and approval history.
Security and governance	No autonomous send; customer-data protection; required disclosure rules outside the model; legal escalation; retain evidence of each material claim.
Operations, evaluation, and recovery	Maximum revisions, regression check for previously passed criteria, independent or diverse evaluator models, and metrics for approval rate, edits, complaints, and citation accuracy.
Required outputs	Approved-ready response, compliance checklist, source citations, unresolved questions, and revision history.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 12.2: SOX control evidence package

Outcome: Assemble control evidence, test completeness against a defined rubric, remediate gaps, and prepare a reviewer-ready package.

Requirement	Detailed specification
Trigger and cadence	Control test window; quarterly certification; audit request; control exception.
Required inputs and data	Control narrative, risk and control matrix, system reports, tickets, approvals, access reviews, timestamps, population definition, sampling rules, and prior findings.
Agent and human roles	Evidence assembler, control-test evaluator, remediation recommender, and human control owner/auditor.
Tools, APIs, and connectors	GRC, ERP, identity governance, ITSM, document repositories, report export, digital signatures, and immutable evidence store.
State and memory	Evidence inventory, source hashes, rubric results, sample selections, gaps, remediation attempts, owner attestations, and final signoff.
Security and governance	Segregation of duties; auditor independence; deterministic sampling and calculations; no synthetic evidence; legal retention; immutable audit trail.
Operations, evaluation, and recovery	Stop when all mandatory criteria pass or escalate; detect duplicate/stale evidence; evaluate reviewer rework, false completeness, and traceability.
Required outputs	Indexed evidence package, test results, exceptions, owner actions, citations to source systems, and approval record.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S2] [S3] [S19]

Mixture of Agents

Also known as: MoA; layered ensemble; worker-aggregator stack

Multiple worker agents independently produce candidate responses. One or more aggregator layers review the candidates and create improved candidates, culminating in a final aggregator. The architecture is feed-forward rather than conversational. [S10]

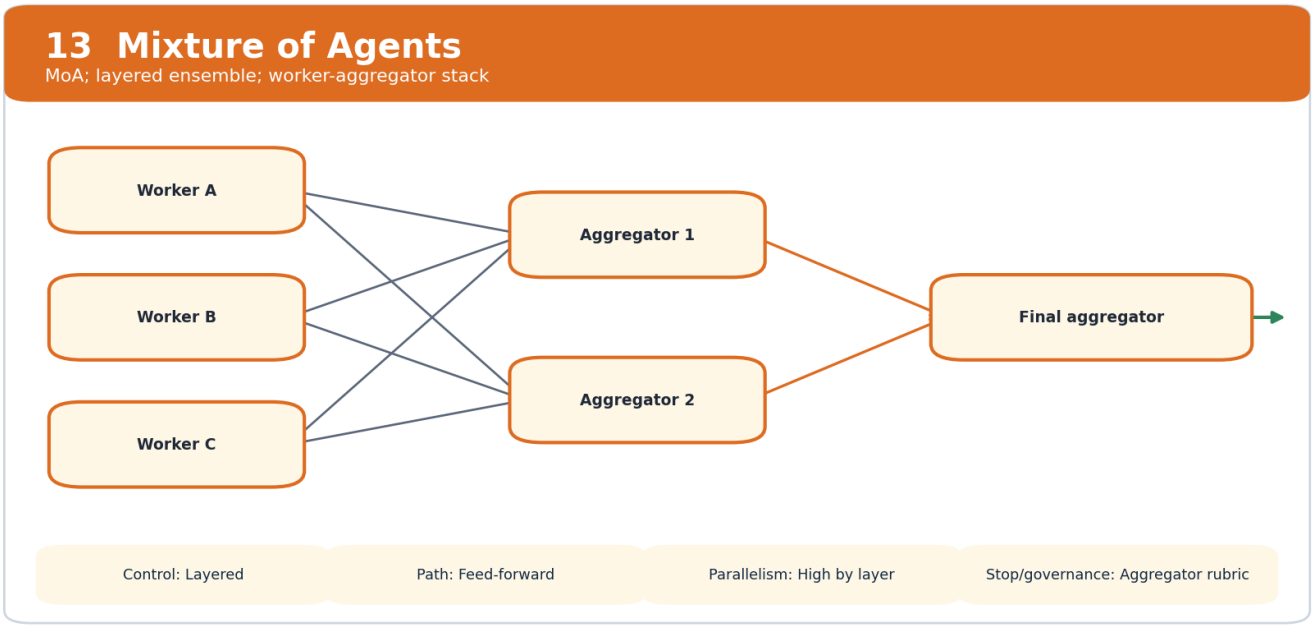


Figure 14. Mixture of Agents - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Layered	**Feed-forward**	**High by layer**	**Medium**	**Aggregator rubric**

USE WHEN

High-value reasoning or synthesis where diverse model perspectives improve quality and extra latency/cost are acceptable.

AVOID WHEN

Tool-heavy transactional work, strict low latency, no reliable aggregation rubric, homogeneous agents that add little diversity, or sensitive prompts that should not be replicated widely.

How it works

1. Generate diverse first-layer candidates
2. Expose candidates to aggregator agents
3. Create one or more improved layers
4. Use a final aggregator to resolve and compose
5. Retain diversity and dissent signals

Use case 13.1: Forecast narrative consensus

Outcome: Generate multiple interpretations of forecast changes and combine them into a balanced, evidence-linked executive narrative.

Requirement	Detailed specification
Trigger and cadence	Forecast refresh; planning meeting; material variance; executive request.
Required inputs and data	Actuals, forecast, pipeline, driver model, assumptions, macro indicators, prior narratives, business-unit commentary, and materiality thresholds.
Agent and human roles	Independent finance, sales, operations, and risk workers; first-layer aggregators by region or business; final executive-narrative aggregator.
Tools, APIs, and connectors	Planning/ERP, CRM, governed semantic model, data warehouse, market-data service, and report workspace.
State and memory	Frozen data snapshot, candidate narratives, evidence links, disagreement matrix, aggregator scores, and selected final language.
Security and governance	Deterministic numbers; no hidden averaging of conflicting facts; finance owner approval; source and model-version logging; limit sensitive data replication.
Operations, evaluation, and recovery	Diversity checks, contradiction detection, weighted aggregation, latency/cost budget by layer, and evaluation of numeric fidelity, bias, and executive usefulness.
Required outputs	Consensus narrative, key drivers, dissent/uncertainty, scenario implications, and citations to the governed data model.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 13.2: Solution-design alternatives

Outcome: Produce genuinely different architecture options and synthesize a recommended design without prematurely converging on one approach.

Requirement	Detailed specification
Trigger and cadence	Architecture discovery; RFP response; modernization decision; design spike.
Required inputs and data	Requirements, constraints, current architecture, workload estimates, security and compliance standards, skills, budget, target platforms, and evaluation rubric.
Agent and human roles	Workers optimized for cost, reliability, security, delivery speed, and platform fit; aggregator architects; final decision synthesizer.
Tools, APIs, and connectors	Architecture repository, cloud pricing/calculators, policy catalog, dependency inventory, diagram generator, and decision-record system.
State and memory	Option artifacts, assumptions, scorecards, conflicts, eliminated options, aggregator feedback, and final rationale.
Security and governance	Human architect owns decision; disclose assumptions; no invented service capabilities or prices; verify current platform limits; preserve rejected alternatives.
Operations, evaluation, and recovery	Require diversity among candidates; common scoring rubric; sensitivity analysis; evaluate factual accuracy, option distinctness, and downstream rework.
Required outputs	Option set, comparison matrix, recommended architecture, risks, cost ranges, assumptions, and architecture decision record draft.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S10] [S3]

Event-Driven / Reactive Agents

Also known as: reactive agents; event-triggered workflow; streaming agent; pub/sub agents

Agents are activated by business, system, or stream events rather than a synchronous user turn. Event Grid suits discrete state-change notifications; Event Hubs suits high-volume ordered telemetry streams. Durable queues, idempotency, retries, and dead-letter handling are central design requirements. [S11] [S16]

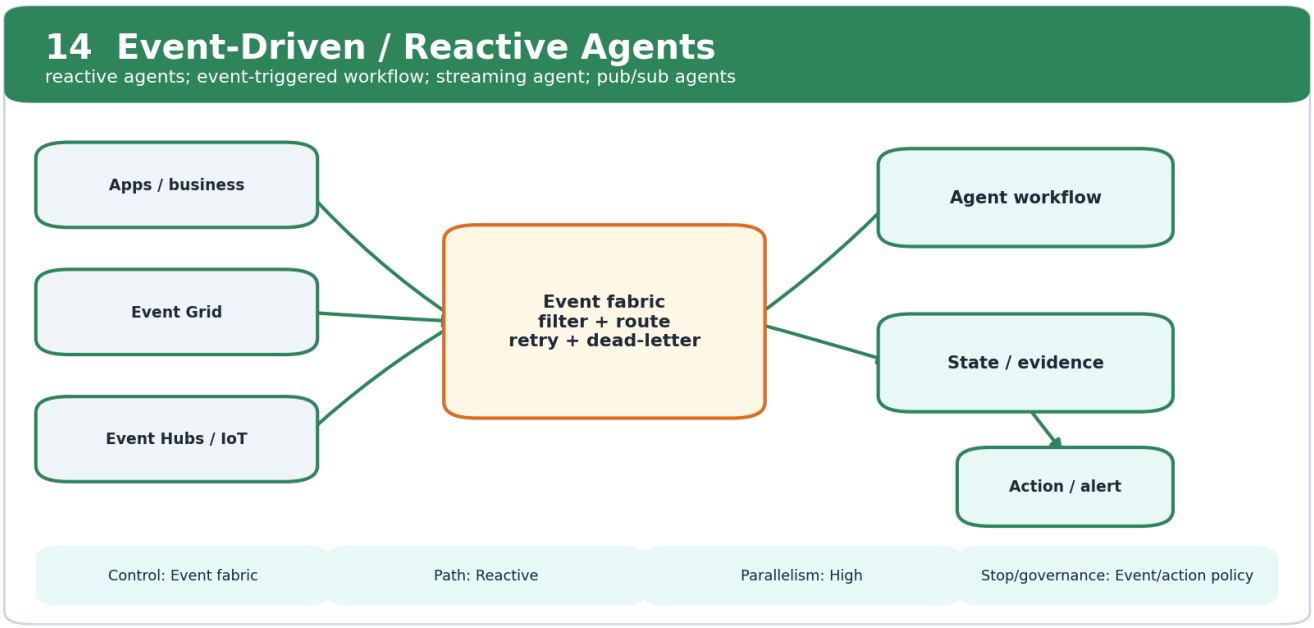


Figure 15. Event-Driven / Reactive Agents - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Event fabric	**Reactive**	**High**	**Variable**	**Event/action policy**

USE WHEN	AVOID WHEN
Continuous monitoring, high-volume telemetry, asynchronous business processes, automation that must react to state changes, or decoupled agent services.	Events lack stable schemas or identity, actions are not idempotent, ordering is critical but unsupported, or human review cannot be accommodated before high-impact action.

How it works

1. Ingest and normalize CloudEvents or stream records
2. Filter, correlate, and deduplicate
3. Invoke the appropriate agent workflow
4. Persist state and publish results/events
5. Retry safely or dead-letter failures

Use case 14.1: Real-time financial-risk and news monitoring

Outcome: Correlate market, news, counterparty, and internal exposure events; summarize material changes; and escalate according to risk policy.

Requirement	Detailed specification
Trigger and cadence	Market tick or threshold; licensed news event; rating change; sanctions update; counterparty exposure change; social-media risk spike.
Required inputs and data	Event Hubs streams for market and transaction data; Event Grid for discrete updates; licensed news/social feeds; exposure and limit data; entity master; watchlists; risk rules.
Agent and human roles	Event classifier, entity-resolution agent, news summarizer, exposure analyst, risk policy agent, and human risk officer.
Tools, APIs, and connectors	Event Hubs, Event Grid, stream analytics, market/news APIs, risk engine, data lake, notification service, case management, and dashboard.
State and memory	Event offsets, deduplication keys, entity graph, rolling exposure, evidence bundle, alert status, and analyst disposition.
Security and governance	Licensed feed use; MNPI and personal-data controls; agents cannot trade; material alerts require human review; immutable alert rationale and source timestamps.
Operations, evaluation, and recovery	Partition strategy, ordering windows, replay, backpressure, idempotency, retry/dead-letter, and latency SLO; evaluate missed events, false positives, and time-to-acknowledge.
Required outputs	Prioritized alert, evidence-linked summary, exposure impact, recommended review steps, and auditable case event.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 14.2: IoT predictive-maintenance response

Outcome: Detect anomalous equipment behavior, correlate it with history and manuals, create maintenance recommendations, and dispatch work under policy.

Requirement	Detailed specification
Trigger and cadence	Sensor anomaly, rule threshold, device fault, missed heartbeat, or maintenance-window event.
Required inputs and data	Event Hubs/IoT telemetry, device metadata, maintenance history, manuals, parts inventory, production schedule, weather/environment, and safety rules.
Agent and human roles	Anomaly interpreter, root-cause agent, maintenance planner, parts agent, scheduling agent, and human maintenance supervisor.
Tools, APIs, and connectors	IoT Hub/Event Hubs, Event Grid, time-series database, CMMS/EAM, inventory, digital twin, notification, and mobile field app.
State and memory	Device health timeline, anomaly window, diagnosis hypotheses, recommended actions, work-order state, and technician feedback.
Security and governance	Safety-critical shutdowns follow deterministic controls; agent recommendation separated from PLC control; technician approval for intrusive work; device identity and network segmentation.
Operations, evaluation, and recovery	Late/out-of-order events, offline buffering, duplicate handling, model drift, fail-safe defaults, and SLOs for detection and dispatch; measure avoided downtime and false alarms.
Required outputs	Maintenance alert, likely cause, parts and skill requirements, proposed work order, schedule, and post-work learning record.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S11] [S16] [S17]

Blackboard / Shared Workspace

Also known as: blackboard; shared memory workspace; artifact-centered collaboration

Agents coordinate by reading and writing facts, hypotheses, artifacts, tasks, and partial solutions to a shared workspace. A scheduler or opportunistic policy selects work based on the current blackboard state. Communication is indirect and artifact-centered rather than a single chat transcript. [S12]

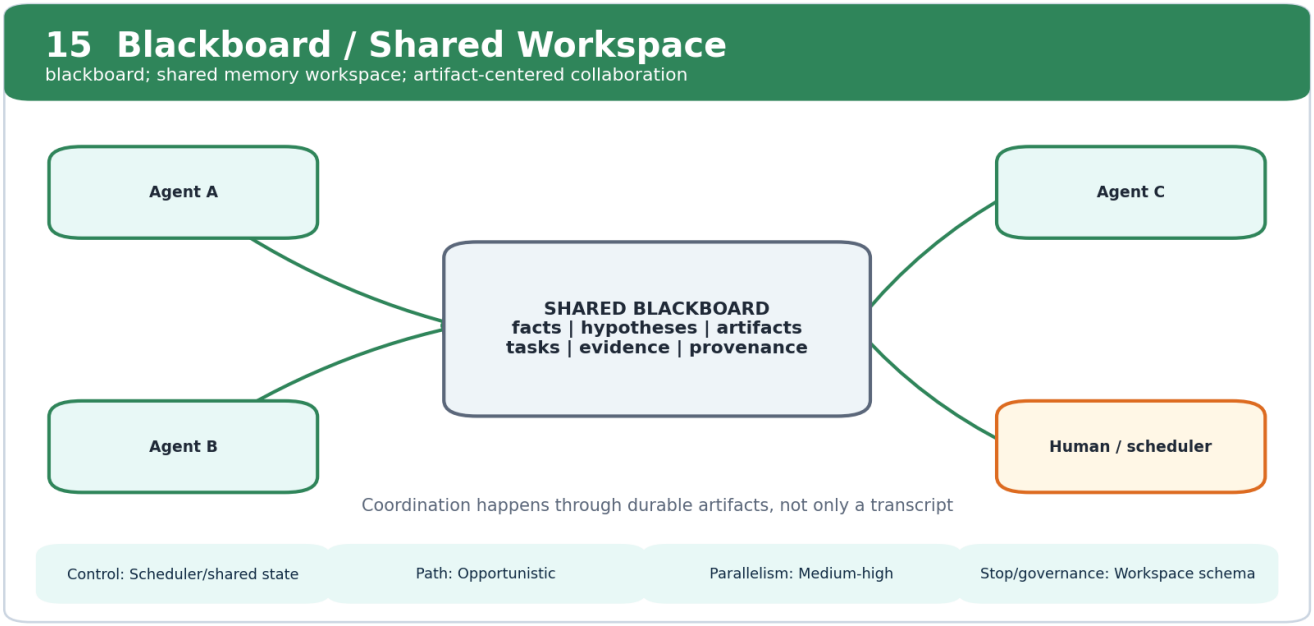


Figure 16. Blackboard / Shared Workspace - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Scheduler/shared state	**Opportunistic**	**Medium-high**	**High**	**Workspace schema**

USE WHEN

Long-lived investigations, case management, scientific or engineering problems, and asynchronous teams that need a durable common operating picture.

AVOID WHEN

Weak provenance, uncontrolled shared writes, simple linear tasks, or sensitive data that cannot be partitioned within the workspace.

How it works

1. Create a shared, typed workspace
2. Let specialists post claims and artifacts
3. Detect new opportunities or conflicts
4. Schedule the next contribution
5. Converge on an accepted workspace state

Use case 15.1: Complex claims or fraud case workspace

Outcome: Allow document, transaction, identity, policy, and investigator agents to build a persistent evidence graph and surface the next best investigative steps.

Requirement	Detailed specification
Trigger and cadence	High-risk claim; anomaly score; investigator request; new evidence or linked-case event.
Required inputs and data	Claim forms, invoices, images, transaction history, identity data, policy terms, prior cases, device/network signals, external records, and investigator notes.
Agent and human roles	Document, policy, transaction, identity, image, network, and timeline agents plus scheduler and human investigator.
Tools, APIs, and connectors	Case system, document intelligence, graph database, fraud models, external-data providers, evidence store, and investigator UI.
State and memory	Typed blackboard objects: facts, claims, evidence, hypotheses, confidence, conflicts, tasks, owners, and dispositions with source-level provenance.
Security and governance	Field-level access; purpose limitation; explainability for adverse decisions; humans own disposition; immutable evidence links; separate allegations from verified facts.
Operations, evaluation, and recovery	Optimistic concurrency, write validation, duplicate/conflict resolution, stale-hypothesis cleanup, and scheduler fairness; evaluate investigator lift and unsupported-claim rate.
Required outputs	Current case picture, evidence graph, prioritized tasks, competing hypotheses, gaps, and investigator-ready recommendation.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 15.2: Product-launch command workspace

Outcome: Maintain one durable operating picture across engineering, marketing, sales, support, legal, and operations as launch conditions evolve.

Requirement	Detailed specification
Trigger and cadence	Launch milestone, telemetry change, readiness update, blocker, customer feedback, or executive question.
Required inputs and data	Project plans, feature readiness, quality metrics, deployment telemetry, content approvals, campaign calendar, seller readiness, support issues, legal constraints, and social feedback.
Agent and human roles	Engineering, quality, operations, marketing, sales-readiness, support, legal, and customer-signal agents plus human launch lead.
Tools, APIs, and connectors	DevOps/GitHub, project tracker, CRM, support, analytics, content repository, social listening, Teams, and status dashboard.
State and memory	Readiness objects, blockers, evidence, decisions, dependencies, owners, due dates, risk thresholds, and launch criteria in a shared schema.
Security and governance	Source-system authority; role-based write permissions; human launch decision; embargo controls; audit of who or what changed readiness status.
Operations, evaluation, and recovery	Event-driven updates, conflict detection, stale-item reminders, schema validation, and rollback of bad writes; measure blocker age, readiness accuracy, and decision latency.
Required outputs	Live readiness view, blocker queue, decision log, risk summary, owner actions, and launch/no-launch recommendation.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S12] [S17]

Swarm / Peer-to-Peer

Also known as: swarm; network; peer agents; decentralized handoff

Peer agents communicate or hand work directly to one another without a permanent central supervisor. Control emerges from local capability rules, shared goals, and stopping policies. The pattern improves local autonomy and resilience but makes global reasoning, loop prevention, and auditability harder. [S13] [S8]

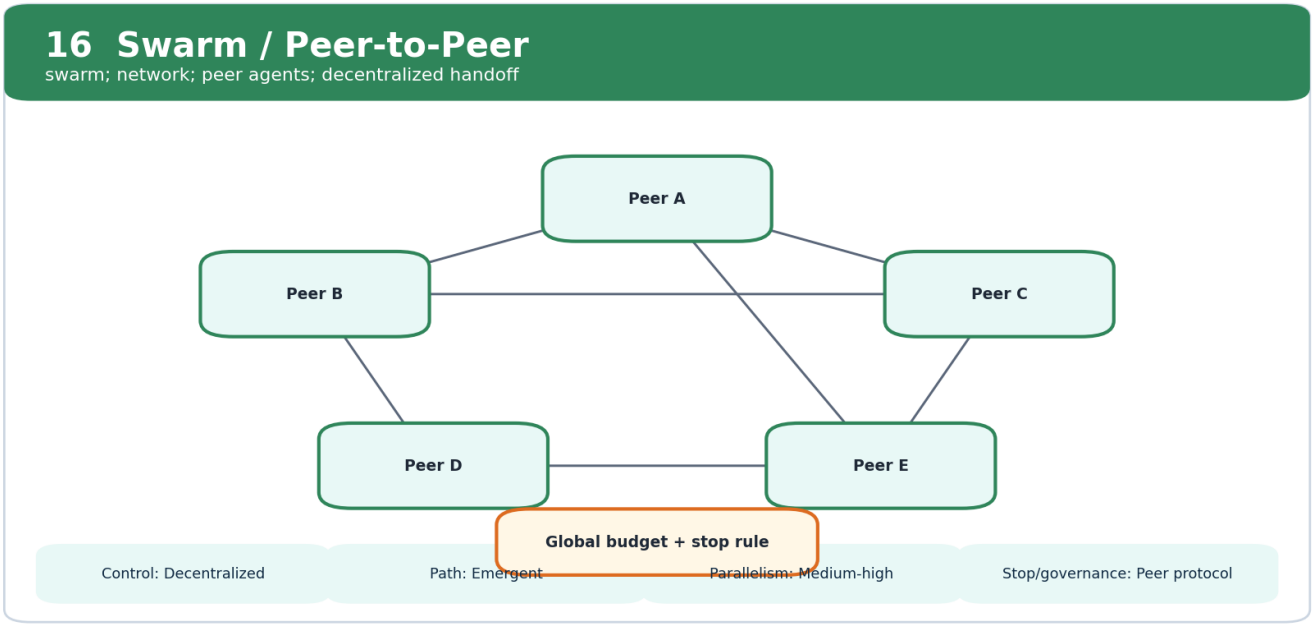


Figure 17. Swarm / Peer-to-Peer - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Decentralized	**Emergent**	**Medium-high**	**High**	**Peer protocol**

USE WHEN

Distributed environments, dynamic specialist availability, local decisions, resilience to a central bottleneck, or simulations of emergent collaboration.

AVOID WHEN

High-stakes centralized control, strict deterministic sequences, unclear global success criteria, or environments lacking strong identity and message governance.

How it works

1. Publish peer capabilities and transfer conditions
2. Let the current peer choose another peer
3. Share minimal state or a common environment
4. Detect loops, dead ends, and duplicate work
5. Terminate through global success or budget rules

Use case 16.1: Distributed cyber threat hunting

Outcome: Let domain peers pursue leads across identity, endpoint, network, cloud, and threat intelligence, handing off as evidence points to another domain.

Requirement	Detailed specification
Trigger and cadence	Threat-hunt hypothesis; new indicator; incident lead; scheduled campaign.
Required inputs and data	Domain telemetry, threat intelligence, asset graph, identity relationships, attack framework, prior hunts, and environment-specific baselines.
Agent and human roles	Identity, endpoint, network, cloud, malware, and threat-intelligence peers with human hunt lead.
Tools, APIs, and connectors	SIEM/XDR, Event Hubs/data lake, graph store, threat-intel platforms, query services, case system, and secure peer messaging.
State and memory	Distributed findings plus shared hunt graph, current lead ownership, evidence, duplicates, handoff trail, and global budget.
Security and governance	Read-only hunting; peer identities and signed messages; query and data-domain scopes; no containment without central/human approval; evidence chain.
Operations, evaluation, and recovery	Loop and duplicate detection, TTL on leads, peer availability, backpressure, quorum for high-confidence findings, and metrics for coverage, novel findings, and wasted work.
Required outputs	Correlated hunt findings, evidence graph, prioritized incidents, unexplored branches, and a complete handoff timeline.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 16.2: Dynamic field-service coordination

Outcome: Coordinate regional dispatch, skills, parts, routing, and site-access peers as conditions change without waiting on one central reasoning agent.

Requirement	Detailed specification
Trigger and cadence	Work order, technician delay, part unavailable, severe-weather event, or priority change.
Required inputs and data	Work orders, technician skills and location, parts inventory, traffic/weather, site access, customer windows, safety requirements, and SLA priority.
Agent and human roles	Regional dispatch, skills, inventory, route, site-access, and safety peers; human dispatcher handles exceptions.
Tools, APIs, and connectors	Field-service platform, GIS/routing, inventory, weather, customer messaging, mobile app, Event Grid, and service bus.
State and memory	Work-order ownership, offers/acceptances, technician route, reservations, constraints, handoffs, and exception history.
Security and governance	Labor and safety rules; location privacy; authorization before customer commitment; signed peer messages; prevent double booking and unauthorized reassignment.
Operations, evaluation, and recovery	Distributed locking or reservation, conflict resolution, offline peer recovery, global SLA monitor, and measures for travel time, first-time fix, and reassignment churn.
Required outputs	Confirmed assignment, route and parts reservation, customer update, exception escalation, and service record.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S13] [S8] [S17]

Human-in-the-Loop (HITL)

Also known as: human approval; request information; escalation; pause-and-resume

The workflow pauses for a person to approve, reject, edit, choose, or supply missing information, then resumes from durable state. HITL is an overlay that can be inserted into any other pattern. It should be risk-based and targeted rather than a vague requirement to review everything. [S1] [S14] [S15]

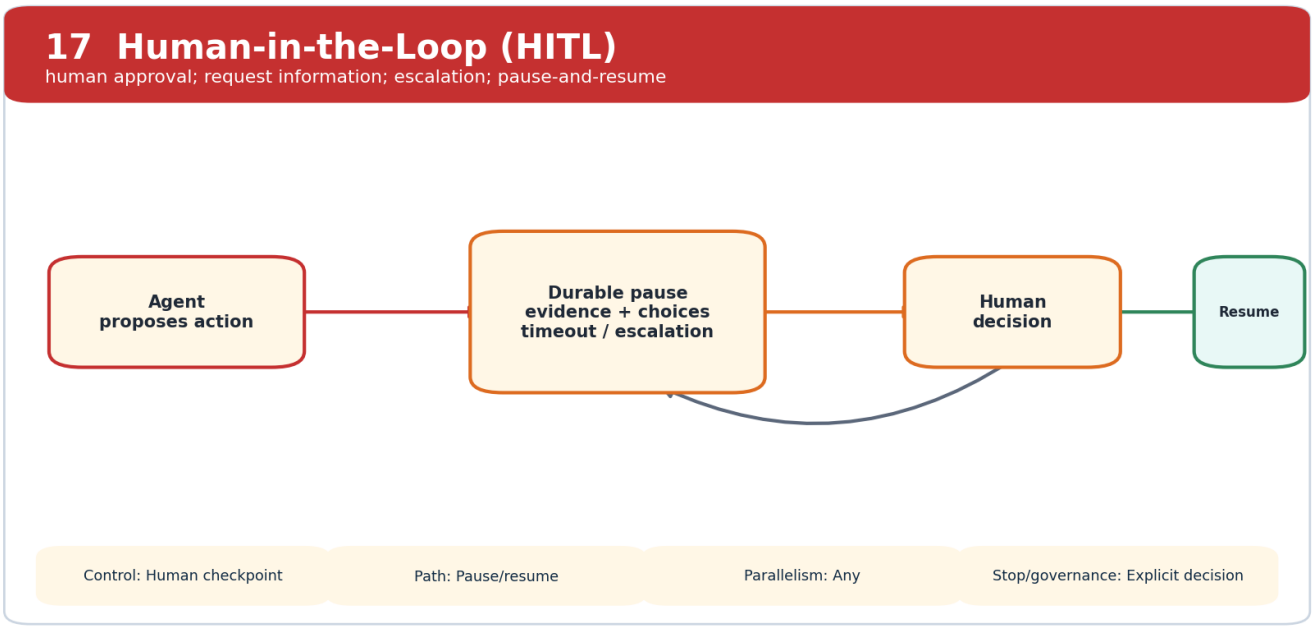


Figure 18. Human-in-the-Loop (HITL) - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Human checkpoint	**Pause/resume**	**Any**	**Risk-bounded**	**Explicit decision**

USE WHEN	AVOID WHEN
High-impact actions, ambiguous evidence, regulated decisions, financial commitments, external publication, access changes, or low-confidence cases.	Approval fatigue, unclear decision rights, non-durable waits, missing context for reviewers, or using a nominal human click to legitimize an unsafe design.

How it works

1. Detect an approval, information, or escalation condition
2. Persist a durable checkpoint
3. Present evidence and bounded choices
4. Authenticate and record the human decision
5. Resume, revise, cancel, or time out safely

Use case 17.1: Procurement and payment approval

Outcome: Prepare a purchase or payment recommendation, validate policy and evidence, and pause for the authorized approver before creating a binding transaction.

Requirement	Detailed specification
Trigger and cadence	Purchase request, invoice received, renewal date, budget threshold, or exception.
Required inputs and data	Request/invoice, vendor master, contract, purchase order, receiving evidence, budget, approval matrix, sanctions check, tax data, and segregation-of-duties rules.
Agent and human roles	Intake, match/validation, policy, risk, and recommendation agents; budget owner, procurement, or finance approver.
Tools, APIs, and connectors	ERP/procurement, inbox/document capture, vendor risk, budget system, identity/approval service, e-signature, and notification channel.
State and memory	Transaction draft, source evidence, match results, exceptions, risk score, approver identity, decision, comments, timeout, and final posting status.
Security and governance	Deterministic three-way match; role and amount-based approval; step-up authentication; no self-approval; immutable decision receipt; dual approval for defined thresholds.
Operations, evaluation, and recovery	Durable wait, reminder and escalation, delegation rules, expiry/revalidation after data changes, and metrics for cycle time, override rate, and policy exceptions.
Required outputs	Approval card with evidence, approved/rejected transaction, exception case, posting receipt, and audit trail.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 17.2: High-risk customer or public communication

Outcome: Draft and check a message, surface claims and risk, and require an accountable person to edit or approve before release.

Requirement	Detailed specification
Trigger and cadence	External email, public post, legal notice, press statement, executive message, or sensitive customer case.
Required inputs and data	Case facts, inbox thread, approved policy and product content, disclosure rules, audience, channel limits, sensitivity label, and source citations.
Agent and human roles	Drafting and checking agents; communications, legal, compliance, account owner, or executive approver based on risk.
Tools, APIs, and connectors	Exchange/CRM/social management, policy retrieval, DLP/sensitivity labels, approval interface, archive, and publication connector.
State and memory	Draft, source facts, risk flags, edits, approver identity, final text, send/publish receipt, and cancellation/timeout status.
Security and governance	No send or publish before approval; reviewer sees changes and evidence; external-recipient confirmation; disclosure and DLP checks; preserve final approved content.
Operations, evaluation, and recovery	SLA-based escalation, mobile approval, stale-draft revalidation, revocation before send, and metrics for edit rate, prevented incidents, and approval latency.
Required outputs	Reviewable message, risk summary, approval decision, final released content, and audit record.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S1] [S14] [S15] [S17]

Adaptive Hybrid / Composite Orchestration

Also known as: adaptive orchestration; composite workflow; pattern-of-patterns

A policy layer selects and composes patterns based on task type, risk, latency, cost, confidence, and available capabilities. For example, a router can choose a sequential workflow, a supervisor can fan out workers, and a HITL gate can protect the final action. This is the normal production shape for broad enterprise systems. [S1] [S2] [S17]

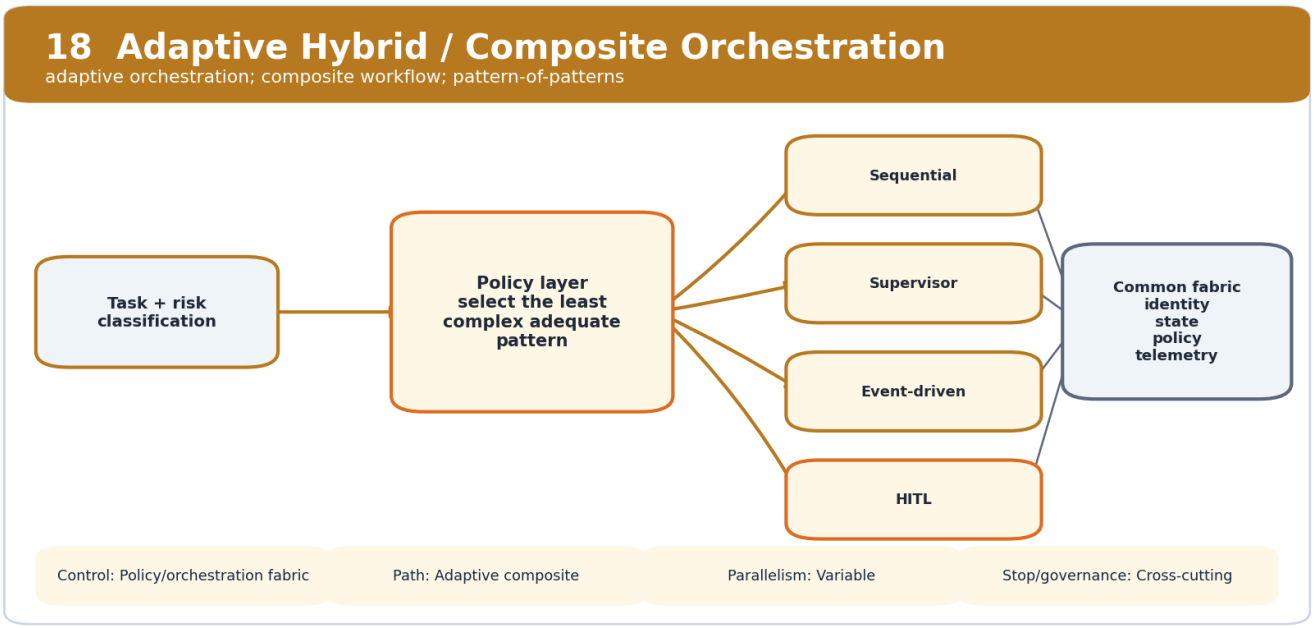


Figure 19. Adaptive Hybrid / Composite Orchestration - conceptual architecture and control characteristics.

CONTROL	PATH	PARALLELISM	AUTONOMY	GOVERNANCE
Policy/orchestration fabric	**Adaptive composite**	**Variable**	**Tiered**	**Cross-cutting**

USE WHEN	AVOID WHEN
Broad digital workers, end-to-end business processes, multiple risk tiers, mixed synchronous and event-driven work, or platforms serving many agent experiences.	Early prototypes without operational maturity, no common state or identity model, uncontrolled framework sprawl, or inability to test the composition as a whole.

How it works

1. Classify task, risk, and service level
2. Select the least complex adequate pattern
3. Compose subworkflows behind typed contracts
4. Apply cross-cutting identity, policy, and telemetry
5. Learn from outcomes without changing controls silently

Use case 18.1: Enterprise digital worker

Outcome: Handle a broad employee request end to end while choosing the simplest suitable pattern and inserting human control only where risk requires it.

Requirement	Detailed specification
Trigger and cadence	Copilot/chat prompt, inbox message, scheduled brief, document upload, or business event.
Required inputs and data	User identity and intent; email, calendar, files, chats, enterprise knowledge, line-of-business data, policies, task context, and risk classification.
Agent and human roles	Front-door router; single-agent resolver for simple tasks; supervisor for multi-domain work; orchestrator-workers for large tasks; event agent; HITL gate; human service desk.
Tools, APIs, and connectors	Microsoft Graph, enterprise search, CRM/ERP/ITSM, workflow runtime, Event Grid/Service Bus, secure tool gateway, identity, policy, and observability platform.
State and memory	Unified task envelope, typed subworkflow state, consent, provenance, pending approvals, action receipts, cost/latency budget, and outcome feedback.
Security and governance	Central capability registry; task-level least privilege; policy-as-code; DLP and labels; approval tiers; framework/version inventory; kill switch and tenant boundaries.
Operations, evaluation, and recovery	Pattern-selection telemetry, end-to-end tracing, graceful fallback to simpler patterns or humans, SLOs by task class, chaos testing, and cost allocation.
Required outputs	Answer, artifact, completed action, approval request, or human handoff with one coherent status and evidence trail.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Use case 18.2: End-to-end customer complaint resolution

Outcome: Ingest a complaint, classify it, investigate in parallel, discuss ambiguous cases, obtain approvals for remedy, communicate, and learn from the outcome.

Requirement	Detailed specification
Trigger and cadence	Inbox message, web form, social post, contact-center transcript, regulator referral, or CRM case event.
Required inputs and data	Customer message and attachments; account and transaction history; product policy; prior contacts; service telemetry; social context; legal/regulatory rules; remedy limits.
Agent and human roles	Router; identity/account, transaction, product, and sentiment agents in parallel; moderator for ambiguity; supervisor; remedy planner; HITL approver; communications agent.
Tools, APIs, and connectors	Exchange/social/contact center, CRM, billing/order systems, telemetry, policy retrieval, Event Grid, case workflow, approvals, outbound channels, and analytics.
State and memory	Case envelope, evidence, routing and orchestration decisions, parallel findings, discussion transcript, proposed remedy, approvals, communication, and disposition.
Security and governance	Customer authentication, privacy and residency, fairness checks, deterministic remedy limits, human approval above thresholds, no public response without communications policy, regulator retention.
Operations, evaluation, and recovery	Correlate duplicate complaints, priority and SLA timers, fallback when a system is unavailable, replay from checkpoints, evaluate first-contact resolution, fairness, cost, and recurrence.
Required outputs	Resolved case, approved remedy, customer response, root-cause signal, product/service action, and complete audit history.

Implementation note. The named products and feeds are examples. Substitute equivalent services while preserving the data contracts, identity boundaries, durability, audit, and evaluation requirements.

Acceptance gates

DATA READY	SAFE TO RUN	OPERABLE
Authoritative sources, access, freshness, schema, and provenance are verified.	Least privilege, action gates, stop rules, and human fallback are tested.	Quality, latency, cost, recovery, and audit targets pass representative tests.

Chapter sources: [S1] [S2] [S16] [S17]

APPENDIX A

Cross-pattern production checklist

Apply these controls regardless of framework or orchestration label. Use the checklist during design review, threat modeling, readiness assessment, and go-live approval.

01. Use least privilege for every agent identity, tool, connector, and data store.
02. Separate read-only analysis tools from tools that create, update, send, pay, delete, or publish.
03. Validate typed inter-agent payloads; reject malformed, oversized, or policy-violating content.
04. Persist correlation IDs, prompt/model/tool versions, decisions, tool calls, costs, and outcomes.
05. Set time, token, tool-call, retry, and orchestration-round budgets with explicit fallback behavior.
06. Evaluate task success, groundedness, safety, latency, cost, and recovery on representative test sets.
07. Treat external content, email, files, social feeds, and tool results as untrusted input; defend against prompt injection.
08. Minimize retained context and apply classification, retention, residency, DLP, and audit requirements.

Data and integration questions

- What is the authoritative source for each fact, calculation, identity, entitlement, and status?
- Is the trigger synchronous, a discrete event (for example Event Grid), a high-volume stream (for example Event Hubs), a scheduled job, or a human request?
- What must be passed between agents, and what must be deliberately withheld?
- Which connectors can only read, which can draft, and which can commit state or communicate externally?
- How are duplicate, late, missing, out-of-order, poisoned, or replayed events handled?
- What is the durable checkpoint and recovery behavior if a model, tool, network, or human approver is unavailable?

Evaluation questions

- What does task success mean, and which dimensions are deterministic versus judgment-based?
- What are the acceptable false-positive and false-negative rates for routing, alerts, compliance, and escalation?
- How will you measure groundedness, numeric fidelity, policy adherence, latency, cost, human rework, and harmful-action prevention?
- What test cases cover prompt injection, data exfiltration, privilege escalation, tool misuse, handoff loops, stale state, and conflicting agents?
- Who reviews failures, owns the rollback or kill switch, and approves changes to prompts, models, tools, policies, and pattern selection?

APPENDIX B

Glossary and distinctions

Term	Working definition
Agent	A model-guided component that can reason over context and invoke tools or knowledge sources to pursue a goal.
Workflow	A primarily predefined control flow. Model calls can be flexible inside steps, while the overall path remains explicit.
Orchestration	The policy and runtime behavior that distribute work, move control, share context, reconcile results, and stop.
Router vs handoff	A router usually chooses a destination at intake; a handoff transfers active ownership during execution.
Supervisor vs moderator	A supervisor delegates to subagents that return results; a moderator manages a shared conversation in which agents see and respond to one another.
Orchestrator-workers vs concurrent	Concurrent work is usually known or selected from a registered set; orchestrator-workers creates input-dependent subtasks at runtime.
Planner-executor vs Magentic	Planner-executor exposes a plan and execution phases; Magentic adds manager-led turn selection, a progress ledger, stall detection, and replanning for open-ended collaboration.
Loop vs evaluator-optimizer	A deterministic loop uses a machine-checkable condition; evaluator-optimizer uses explicit qualitative criteria and feedback.
Event Grid vs Event Hubs	Event Grid distributes discrete events and state changes. Event Hubs ingests high-throughput ordered streams and telemetry.
HITL	A durable human decision, information, or escalation point layered into any pattern.
MCP and A2A	Protocols that can support tools/context and agent-to-agent communication. They are interoperability mechanisms, not orchestration patterns by themselves.

REFERENCES

Authoritative and primary sources

Accessed 22 October 2026. Source dates are included when exposed by the publisher. Product features, preview status, limits, and naming can change; verify implementation details against the current documentation before deployment.

[S1] Microsoft Learn. Workflow orchestrations in Agent Framework. Built-in sequential, concurrent, handoff, group chat, and Magentic orchestrations; HITL support. Updated 25 August 2026.

[S2] Azure Architecture Center. AI Agent Orchestration Patterns. Pattern selection guidance, tradeoffs, aliases, and implementation considerations.

[S3] Anthropic. Building Effective AI Agents. Prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer, and autonomous agents.

[S4] Google Developers Blog. Developer's guide to multi-agent patterns in ADK. Sequential, parallel, loop, hierarchical delegation, and transfer patterns. Published 16 December 2025.

[S5] Microsoft Learn. Microsoft Agent Framework Workflows Orchestrations - Group Chat. Shared conversation, manager selection, and human participation.

[S6] Microsoft Learn. Microsoft Agent Framework Workflows Orchestrations - Magentic. Dynamic manager, progress ledger, replanning limits, streaming, and optional plan signoff.

[S7] AutoGen. Magentic-One. Generalist multi-agent system for open-ended web and file tasks.

[S8] LangChain Documentation. Multi-agent. Subagents, handoffs, skills, router, and custom workflow patterns.

[S9] Amazon Bedrock Documentation. Create multi-agent collaboration. Supervisor and collaborator-agent implementation model.

[S10] AutoGen. Mixture of Agents. Layered worker agents and aggregators inspired by feed-forward architectures.

[S11] Microsoft Learn. Develop Event-Driven AI Workflows with Azure Event Grid. Reactive AI workflows, CloudEvents, delivery policies, retries, and dead-lettering. Updated 27 July 2026.

[S12] arXiv. Exploring Advanced LLM Multi-Agent Systems Based on Blackboard Architecture. Shared blackboard, role-specialized agents, and global information visibility. Published July 2025.

[S13] AWS Builder Center. Peer-to-Peer Agent Swarm with Strands Agents. Decentralized peer collaboration and direct handoffs.

[S14] Microsoft Learn. Microsoft Agent Framework Workflows - Human-in-the-loop (HITL). Pause/resume, request-information, and approval patterns. Updated 30 July 2026.

[S15] Microsoft Foundry Documentation. Add a human-in-the-loop approval step. Durable approval checkpoints and resume behavior. Updated 18 August 2026.

[S16] Azure Architecture Center. Dynamic AI Agents at Scale Pattern. Scalable runtime, messaging, persistence, networking, and observability considerations. Updated 21 July 2026.

[S17] Microsoft Learn Agents Architecture. Multi-agent patterns. Typed payloads, context minimization, parallelism, user participation, approvals, and reconciliation. Updated 5 June 2026.

[S18] AutoGen. Group Chat. Shared-topic group chat with specialized participant agents and a manager.

[S19] LangGraph Documentation. Workflows and agents. Routing, parallelization, orchestrator-worker, and evaluator-optimizer implementations.